학습 교안

데이터 분석 프로젝트 | 노트북 01 · 02 · 03 보충 교안

Olist 이커머스 데이터 분석

분석에서 웹 서비스 배포까지

데이터 구조 · 분석 기법 · 주요 인사이트 10가지 · Streamlit 대시보드 · GitHub · Streamlit Cloud 배포 · 내 도메인으로 확장하기

이 교안만 따라 하면 누구나 자기 이름의 대시보드 주소(...streamlit.app)를 만들 수 있도록 실제 화면과 함께 단계별로 설명합니다.

구성

1~2장 프로젝트 한눈에 보기 · 실습 환경과 데이터 준비

3~4장 데이터 구조 이해 · 분석 핵심 기법과 흔한 실수

5장 이 데이터로 찾을 수 있는 주요 인사이트 10가지

6장 Streamlit 대시보드 이해하기 (app.py 해부)

7~9장 GitHub 업로드 · Streamlit Cloud 배포 · 문제 해결 — 실제 화면으로 따라 하기

10장 내 도메인 지식으로 확장하기 — 다른 분야 데이터로 나만의 서비스 만들기

부록 확인 문제 정답 · 명령어 요약 · 배포 전 체크리스트

예시 저장소 : https://github.com/sinaboro/Olist-E-Commerce-Data-Analysis

예시 서비스 : https://olist-ecommerce-sinaboro.streamlit.app

목 차

이 교안의 사용법 5

프로젝트 한눈에 보기 6

1.1 무엇을 만드나요? 6

1.2 완성 화면 7

1.3 폴더 구조 7

실습 환경과 데이터 준비 9

2.1 설치할 것 9

2.2 저장소 받기 — 데이터가 함께 들어 있어요 9

2.3 환경 확인 10

데이터 구조 이해하기 12

3.1 테이블 관계도 12

3.2 1 : N 관계와 merge — 행이 늘어나는 함정 13

3.3 분석 대상 정의 — 어떤 주문을 '매출'로 볼까? 13

분석 핵심 기법과 흔한 실수 14

4.1 핵심 기법 7가지 14

4.2 월 단위 묶기 — ME 와 MS 의 차이 14

4.3 공정한 비교와 대표값 14

4.4 흔한 실수 6가지 15

주요 인사이트 10가지 16

인사이트 1. 고객 대부분이 한 번 사고 떠난다 16

인사이트 2. 고객은 남동부 3개 주에 몰려 있다 16

인사이트 3. 신용카드 할부가 구매의 기본이다 17

인사이트 4. '보통 주문'은 평균보다 훨씬 작다 18

인사이트 5. 성장은 객단가가 아니라 주문 수에서 왔다 18

인사이트 6. 블랙프라이데이 — 그리고 공정한 비교의 중요성 19

인사이트 7. 평일 낮에 사고, 새벽 · 주말에는 덜 산다 20

인사이트 8. 많이 팔리는 것과 돈이 되는 것은 다르다 20

인사이트 9. 배송이 늦으면 리뷰가 무너진다 (가장 중요) 21
인사이트 10. 주문이 몰리면 지연이 늘고, 먼 지역은 더 오래 걸린다 22
5.11 인사이트를 실행으로 — 한 장 요약 23
Streamlit 대시보드 이해하기 25
6.1 Streamlit 의 원리 — 조작할 때마다 다시 실행 25
6.2 app.py 의 구조 25
6.3 탭별 화면 26
6.4 내 컴퓨터에서 실행하기 28
GitHub 에 올리기 30
7.1 git 과 GitHub 30
7.2 (처음 한 번) GitHub 가입과 git 설정 30
7.3 GitHub 에 빈 저장소 만들기 31
7.4 내 컴퓨터에서 commit 하고 push 하기 31
Streamlit Cloud 로 배포하기 34
8.1 접속하고 로그인하기 34
8.2 (처음 한 번) 계정 정보 입력 34
8.3 GitHub 연결하기 35
8.4 앱 만들기 — 저장소 · 브랜치 · 파일 · 주소 36
8.5 Python 버전 맞추기 (Advanced settings) 38
8.6 Deploy — 서비스 시작 39
8.7 앱 관리하기 40
8.8 README 에 주소 적기 41
8.9 고치고 다시 배포하기 — push 만 하면 끝 41
문제 해결 42
배포 전 체크리스트 42
내 도메인 지식으로 확장하기 44
10.1 분석 서비스는 7단계를 반복한다 44
10.2 이커머스 개념을 일반화하기 44
10.3 도메인별 확장 예시 45
10.4 KPI 정의서 쓰기 45

10.5 데이터 구하기 46
10.6 app.py 를 내 데이터로 바꾸는 순서 46
10.7 미니 프로젝트 — 나만의 분석 서비스 47
10.8 더 나아가기 48
부록 A. 확인 문제 정답 49
1장 프로젝트 한눈에 보기 49
2장 실습 환경과 데이터 준비 49
3장 데이터 구조 이해하기 49
4장 분석 핵심 기법과 흔한 실수 50
5장 주요 인사이트 10가지 50
6장 Streamlit 대시보드 이해하기 50
7장 GitHub 에 올리기 50
8장 Streamlit Cloud 로 배포하기 50
9장 문제 해결 51
10장 내 도메인 지식으로 확장하기 51
부록 B. 명령어 · 코드 한 장 요약 52
부록 C. 참고 링크 53

START

이 교안의 사용법

노트북 3개와 함께 보는 보충 교안

이 교안은 예시 저장소의 **notebooks** 폴더에 있는 노트북 3개를 **보충**합니다. 노트북에는 코드와 실행 결과가, 이 교안에는 그 코드를 이해하는 데 필요한 배경 설명 · 그림 · 실제 서비스 화면 · 문제 해결 방법이 담겨 있습니다. 특히 6~9장은 **이 교안만 보고도** 대시보드를 인터넷에 공개할 수 있도록 모든 화면을 순서대로 실었습니다.

노트북	내용	이 교안에서 함께 볼 장
01_데이터이해_고객_매출	테이블 구조, 분석 대상 정의, 고객 · 재구매 · 지역 · 결제, 객단가, 월별 매출	3장 · 4장 · 5장(인사이트 1~6)
02_시간패턴_카테고리_배송리뷰	요일 · 시간 패턴, 공정한 비교, 카테고리, 배송 · 지연 · 리뷰, 지역별 배송	4장 · 5장(인사이트 6~10)
03_대시보드_만들기와_GitHub_배포	배포용 데이터, app.py, 테스트, git · GitHub, Streamlit Cloud	6장 · 7장 · 8장 · 9장

표시	의미
[코드]	노트북이나 app.py 에 쓰는 파이썬 코드 (초록 글자 = 주석)
[터미널]	VS Code 터미널 · 명령 프롬프트에 입력하는 명령
그림 n-n	실제 데이터로 그린 그래프, 또는 실제 서비스 화면을 캡처한 사진
학습 목표(파랑) · 핵심(파랑 굵게)	장의 목표 / 꼭 기억할 한 문장
주의(노랑) · 오류(빨강)	자주 하는 실수 / 실제 오류 메시지와 해결법
더 알아보기(회색) · 도전 과제(초록)	조금 더 깊은 내용 / 스스로 해 볼 과제

완성본 미리 보기

- 예시 GitHub 저장소: **https://github.com/sinaboro/Olist-E-Commerce-Data-Analysis**
- 예시 대시보드(서비스): **https://olist-ecommerce-sinaboro.streamlit.app**
- 이 교안을 끝까지 따라 하면 여러분의 GitHub 계정에 같은 저장소가, 여러분이 정한 이름으로 같은 대시보드 주소가 생깁니다.

CHAPTER 1

프로젝트 한눈에 보기

분석 → 대시보드 → GitHub → Streamlit Cloud → 누구나 접속

학습 목표

- 이 프로젝트의 전체 흐름과 각 단계의 결과물을 설명할 수 있다.
- 분석(노트북)과 서비스(app.py)가 어떻게 연결되는지 이해한다.

1.1 무엇을 만드나요?

브라질 온라인 마켓플레이스 **Olist** 의 실제 주문 데이터(2016~2018년, 약 10만 건)를 분석해 인사이트를 찾고, 그 결과를 **웹 대시보드**로 만들어 인터넷에 공개합니다. 분석 결과를 보고서 파일로 보내는 대신 **주소 하나로 누구나 필터를 바꿔 가며 볼 수 있는 서비스**로 만드는 것이 목표입니다.

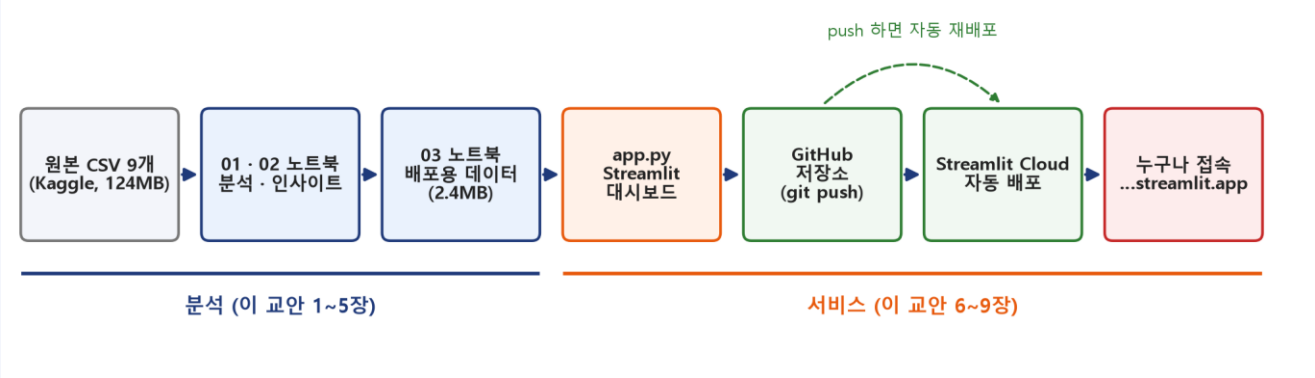


그림 1-1 프로젝트 전체 흐름 — 왼쪽 3단계는 분석, 오른쪽 4단계는 서비스

단계	도구	결과물
① 데이터 준비	Kaggle → 저장소	data 폴더의 원본 9개 (압축 .csv.gz, 126MB → 44MB, 저장소에 포함)
② 탐색적 분석	pandas · plotly (노트북 01 · 02)	인사이트 10가지 (5장)
③ 배포용 가공	pandas (노트북 03)	data/orders.csv.gz · items.csv.gz (약 2.4MB)
④ 대시보드	Streamlit (app.py)	내 컴퓨터에서 열리는 웹 앱 (localhost:8501)
⑤ 버전 관리 · 공유	git · GitHub	github.com/내아이디/저장소이름
⑥ 배포	Streamlit Community Cloud (무료)	https://내가정한이름.streamlit.app

1.2 완성 화면

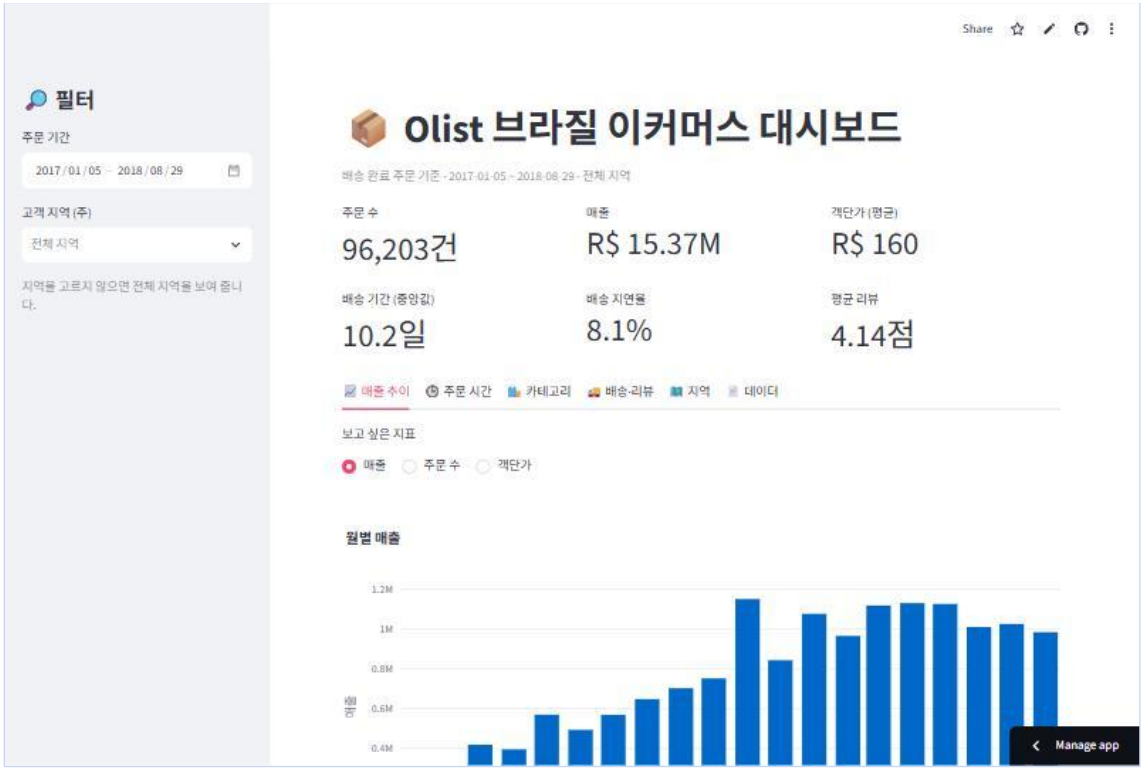


그림 1-2 배포된 대시보드 — 왼쪽 필터, 위쪽 핵심 지표 6개, 아래 6개 탭

1.3 폴더 구조

[폴더 구조]

```
Olist-E-Commerce-Data-Analysis/       ← 저장소 (내 컴퓨터에 받아 둔 폴더)
├── data/
│   ├── olist_….csv.gz 8개 + product_category_name_translation.csv.gz
│   │                                 ← Kaggle 원본 9개를 압축한 것 (126MB → 44MB) – 저장소에 포함
│   ├── orders.csv.gz · items.csv.gz  ← 대시보드용 가공 데이터 (2.4MB) – 저장소에 포함
├── notebooks/
│   ├── 01_데이터이해_고객_매출.ipynb        ← PATH = "../data/" 로 원본을 읽음
│   ├── 02_시간패턴_카테고리_배송리뷰.ipynb
│   ├── 03_대시보드_만들기와_GitHub_배포.ipynb
│   └── olist_dashboard/              ← 03 노트북을 실행하면 생김 = 내 GitHub 에 올릴 폴더
│       ├── app.py · requirements.txt · README.md · .gitignore · .streamlit/
│       └── data/  orders.csv.gz · items.csv.gz
├── app.py · requirements.txt · .streamlit/   ← olist_dashboard 와 같은 내용 (예시 서비스가 쓰는 파일)
└── docs/  이 교안 PDF
```

핵심

노트북은 '분석하는 곳', app.py 는 '보여 주는 곳', GitHub 는 '보관하는 곳', Streamlit Cloud 는 '실행해 주는 곳'이다.

■ 확인 문제 (정답은 부록 A)

1. 대시보드가 원본 데이터(압축 44MB) 대신 2.4MB 짜리 가공 데이터를 읽는 이유 두 가지는?

2. Streamlit Cloud 는 어디에 있는 코드를 실행하나요?

CHAPTER 2

실습 환경과 데이터 준비

Python · VS Code · 패키지 설치 · 저장소 받기 · 데이터 확인

학습 목표

- 실습에 필요한 프로그램과 패키지를 설치한다.
- 예시 저장소를 받고 data 폴더의 데이터(.csv.gz)를 확인한다.
- 노트북 첫 셀을 실행해 환경을 확인한다.

2.1 설치할 것

프로그램	확인 방법 (터미널)	설치
Python 3.10 이상 (3.12 권장)	python --version	python.org → Downloads (설치 화면에서 'Add python.exe to PATH' 체크)
VS Code	-	code.visualstudio.com → 설치 후 확장(Extensions)에서 Python · Jupyter 설치
git	git --version	git-scm.com → Download for Windows → 설치 옵션은 기본값 그대로
패키지	pip list	아래 명령 한 줄

[터미널]

```
pip install pandas plotly streamlit jupyter ipykernel
```

터미널 여는 법

VS Code 메뉴 **보기 → 터미널**(단축키 Ctrl + `) 을 누르면 아래쪽에 터미널이 열립니다. 이 교안의 [터미널] 상자는 모두 여기에 입력합니다. 명령을 입력한 뒤 **Enter** 를 누르세요.

2.2 저장소 받기 — 데이터가 함께 들어 있어요

예시 저장소를 내 컴퓨터로 받습니다. 저장소 페이지에서 초록 **Code** 버튼 → **Download ZIP** 으로 받아 압축을 풀거나, git 을 쓸 줄 알면 아래 명령으로 받습니다. **원본 데이터 9개가 data 폴더에 압축(.csv.gz)되어 함께 들어 있으므로 따로 내려받을 필요가 없습니다.**

[터미널]

```
git clone https://github.com/sinaboro/Olist-E-Commerce-Data-Analysis.git
```

[data 폴더에 들어 있는 파일]

```
olist_customers_dataset.csv.gz          olist_order_reviews_dataset.csv.gz
olist_geolocation_dataset.csv.gz        olist_orders_dataset.csv.gz
olist_order_items_dataset.csv.gz        olist_products_dataset.csv.gz
olist_order_payments_dataset.csv.gz     olist_sellers_dataset.csv.gz
product_category_name_translation.csv.gz
orders.csv.gz · items.csv.gz            ← (이 두 개는 대시보드용 가공 데이터)
```

- **data** 폴더는 노트북 폴더(notebooks)의 **한 단계 위**, 즉 저장소 최상위에 있습니다. 노트북 코드가 `PATH = "../data/"` 로 읽기 때문입니다.
- 파일 이름이 **.csv.gz** 로 끝나면 pd.read_csv 가 압축을 풀지 않고 그대로 읽습니다. 원본 126MB 를 44MB 로 줄여 GitHub 의 50MB 경고를 피하고, clone 과 배포도 빨라집니다.
- 엑셀로 직접 열어 보고 싶다면 반디집 · 7-Zip 으로 압축을 풀면 됩니다. 압축을 푼 .csv 는 .gitignore 의 **data/*.csv** 규칙 때문에 git add . 를 해도 저장소에 올라가지 않습니다.

더 알아보기 — 원본은 어디서 왔나요?

Kaggle「Brazilian E-Commerce Public Dataset by Olist」(kaggle.com/datasets/olistbr/brazilian-ecommerce) 의 CSV 9개를 **내용 변경 없이 압축만** 한 것입니다. 최신 원본이 필요하면 Kaggle 에 로그인해 Download 로 받을 수 있습니다.

데이터 라이선스

Olist 데이터는 **CC BY-NC-SA 4.0** 입니다. 비영리 목적으로 쓰고, 출처를 표시하고, 가공한 결과도 같은 조건으로 공유해야 합니다. 대시보드 하단과 README 에 출처를 적는 이유입니다.

2.3 환경 확인

VS Code 에서 받은 **저장소 폴더**를 열고(파일 → 폴더 열기) notebooks 폴더의 01 노트북을 엽니다. 오른쪽 위 **커널 선택**에서 설치한 Python 을 고른 뒤 첫 두 셀을 실행해 오류가 없는지 확인합니다.

증상	원인과 해결
ModuleNotFoundError: No module named 'plotly'	커널로 고른 Python 에 패키지가 없음 → 그 Python 의 터미널에서 pip install 다시 실행, 또는 커널을 바꿈
FileNotFoundError: ../data/olist_...csv.gz	노트북을 notebooks 폴더 밖으로 옮겼거나 data 폴더가 없음 → 저장소 구조 그대로(notebooks 의 한 단계 위에 data) 두었는지 확인
'python' 은(는) 내부 또는 외부 명령... 이 아닙니다	설치할 때 PATH 추가를 안 함 → Python 재설치하며 'Add python.exe to PATH' 체크
그래프가 안 보임	VS Code 의 Jupyter 확장이 오래됨 → 확장 업데이트 후 VS Code 재시작

■ 확인 문제 (정답은 부록 A)

1. 노트북 코드의 PATH = "../data/" 에서 .. 은 무엇을 뜻하나요?

2. 원본 데이터를 .csv 가 아니라 .csv.gz 로 저장소에 두는 이유는?

3. Olist 데이터를 사용할 때 지켜야 할 라이선스 조건 세 가지는?

CHAPTER 3

데이터 구조 이해하기

노트북 01 PART A 보충 — 테이블 9개와 연결 고리(키)

학습 목표

- 9개 테이블이 어떤 키로 연결되는지 그림으로 설명할 수 있다.
- 1 : N 관계에서 merge 하면 행이 늘어나는 이유를 안다.
- 분석 대상(배송 완료 주문)을 왜 먼저 정하는지 이해한다.

3.1 테이블 관계도

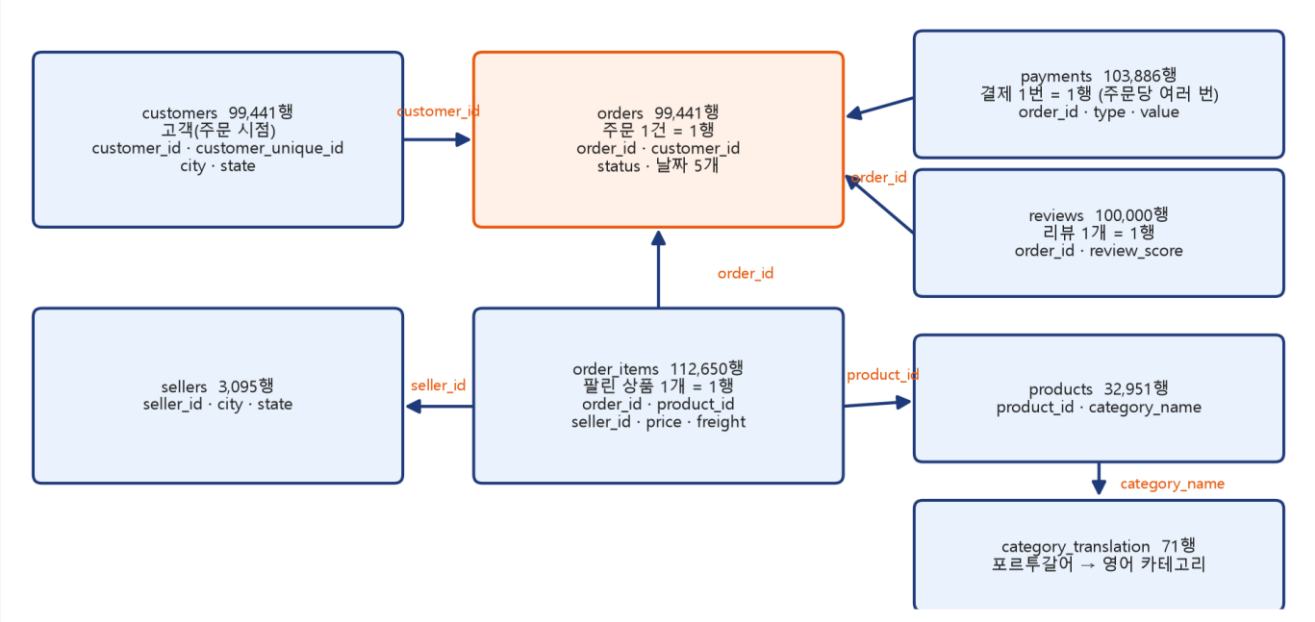


그림 3-1 Olist 데이터의 테이블 관계 — 가운데 orders 를 중심으로 연결된다 (주황 글자 = 연결 키)

키	뜻	주의할 점
order_id	주문 번호	orders 에서는 한 번씩, payments · items · reviews 에서는 여러 번 나올 수 있음
customer_id	주문할 때마다 새로 생기는 고객 번호	같은 사람도 주문마다 다름 → 고객 수를 세면 안 됨
customer_unique_id	실제 사람 한 명의 번호	고객 수 · 재구매는 이것으로 센다 (96,096명)
product_id · seller_id	상품 · 판매자 번호	items 를 거쳐야 주문과 연결됨

3.2 1 : N 관계와 merge — 행이 늘어나는 함정

주문 1건에 결제가 여러 번(최대 29번) 있을 수 있습니다. orders(1) 와 payments(N) 를 그대로 merge 하면 결제 수만큼 주문 행이 **복제**됩니다. 이 상태에서 행 수를 세면 '주문 수'가 아니라 '결제 수'가 됩니다.

[코드]

```
# 잘못된 방법: 결제 3번 한 주문은 3행이 됨 → count() 가 주문 수보다 커짐
wrong = pd.merge(orders, payments, on="order_id")


# 올바른 방법: payments 를 주문별로 먼저 합친 뒤(1행으로) merge
order_value = payments.groupby("order_id", as_index=False)["payment_value"].sum()
sales = pd.merge(delivered, order_value, on="order_id")
sales["order_id"].is_unique        # True 인지 꼭 확인!
```

핵심

merge 하기 전에 키가 '한 번씩만' 나오는지(is_unique) 확인하고, N 쪽 표는 먼저 groupby 로 묶는다.

3.3 분석 대상 정의 — 어떤 주문을 '매출'로 볼까?

orders 의 order_status 는 delivered(배송 완료 96,478건) 외에 shipped · canceled · unavailable 등이 있습니다. 배송 완료일이 빈 칸인 주문은 대부분 **아직 끝나지 않은 주문**입니다. 그래서 이 프로젝트는 dropna() 로 빈 칸을 지우는 대신 **order_status == "delivered"** 로 분석 대상을 명확히 정합니다.

주의 — dropna() 가 뜻하는 것

orders.dropna() 는 '빈 칸이 하나라도 있는 행 삭제'라서, 배송은 끝났는데 승인 시각만 빠진 주문 23건까지 지우고, 반대로 취소됐지만 날짜가 다 있는 주문 6건은 남깁니다. **기준을 말로 설명할 수 있는 필터**를 쓰세요.

주의 — 데이터의 처음과 끝

2016년 9~12월과 2018년 9~10월은 주문이 거의 없습니다(데이터 수집 기간의 경계). 월평균 · 연도 비교에 넣으면 결론이 왜곡되므로 **2017년 1월 ~ 2018년 8월(20개월)** 을 기준 기간으로 씁니다.

■ 확인 문제 (정답은 부록 A)

1. 고객 수를 셀 때 customer_id 가 아니라 customer_unique_id 를 쓰는 이유는?
2. orders 와 payments 를 그대로 merge 한 뒤 행을 세면 무엇이 세어지나요?
3. 이 프로젝트에서 '매출'로 보는 주문의 조건은?

CHAPTER 4

분석 핵심 기법과 흔한 실수

노트북 01 · 02 보충 — 꼭 알아야 할 pandas 기법 7가지

학습 목표

- 노트북에서 쓴 핵심 pandas 기법의 사용법과 함정을 정리한다.
- 공정한 비교와 알맞은 대표값을 고르는 기준을 안다.

4.1 핵심 기법 7가지

기법	코드 예	이럴 때 쓴다
merge 의 how	pd.merge(a, b, on="키", how="left")	inner: 양쪽 모두 있는 행만 / left: 왼쪽 행은 모두 유지 (카테고리 번역이 없는 상품도 살리기)
named aggregation	df.groupby("그룹").agg(매출=("price", "sum"), 주문=("order_id", "nunique"))	여러 계산을 한 번에, 결과 열 이름까지 지정
to_datetime	pd.to_datetime(s, format="%Y-%m-%d %H:%M:%S")	글자 날짜 → 날짜형 (월별 묶기 · 날짜 빼기 가능)
Grouper / resample	df.groupby(pd.Grouper(key="ts", freq="MS"))	월 · 일 단위로 묶기. 그래프용은 MS(월초) 권장
.dt 접근자	s.dt.dayofweek · s.dt.hour · s.dt.to_period("M")	요일(월=0) · 시 · 연-월 꺼내기
pivot_table	df.pivot_table(index="요일", columns="시", values="order_id", aggfunc="count")	긴 표 → 2차원 표 (히트맵 재료)
pd.cut	pd.cut(days, bins=[0, 7, 14, 21, 30], labels=[...])	숫자를 구간(범주)으로 나누기

4.2 월 단위 묶기 — ME 와 MS 의 차이

freq="ME"(월말) 로 묶으면 11월 매출이 **11월 30일** 날짜에 찍혀, plotly 그래프에서 x축의 '12월' 눈금 바로 옆에 막대가 그려집니다. 그래프로 그릴 때는 **freq="MS"(월초, 11월 1일)** 로 묶어야 눈금과 막대가 정확히 맞습니다.

4.3 공정한 비교와 대표값

- **공정한 비교** — 데이터 기간이 그룹마다 다르면 합계를 비교하면 안 됩니다. 2016년(2개월) · 2018년(8개월)처럼 기간이 다를 때는 '같은 1~8월끼리' 비교하거나 '월평균'으로 비교합니다(5장 인사이트 6).
- **대표값** — 주문 금액 · 배송 기간처럼 한쪽으로 꼬리가 긴 데이터는 평균이 극단값에 끌려갑니다. **평균과 중앙값을 함께** 보고, 대표값으로는 보통 **중앙값**을 씁니다.

- **이상치 처리** — 이상치를 지우기보다 중앙값처럼 이상치에 강한 방법을 먼저 고려합니다. 지울 때는 기준(예: 상위 5%)과 이유를 기록합니다.

4.4 흔한 실수 6가지

실수	증상	올바른 방법
payments 를 묶지 않고 merge	주문 수가 4% 정도 부풀려짐 (2017년 11월: 7,592 vs 실제 7,288)	groupby("order_id") 로 먼저 합치기
groupby().sum() 에서 열을 안 고름	결제 수단 글자가 'vouchercredit_card' 처럼 이어 붙음	["payment_value"].sum() 처럼 열 지정
기간이 다른 그룹의 합계 비교	4분기가 꼴찌처럼 보임 (실제로는 2017년 4분기가 1위)	같은 기간끼리 · 월평균 비교
판매 '수량'을 '주문 수'라고 표기	bed_bath_table 11,000개 ≠ 9,300건	size(수량)와 nunique(주문 수) 구분
그래프 결과로 원인을 단정	'물류 개선 덕분' 같은 근거 없는 주석	관찰한 사실만 쓰고 원인은 추가 분석
노트북 마크다운에 R$ 를 두 번 씀	$ … $ 사이가 수식으로 바뀌어 글자가 깨짐	'160헤알' 로 쓰거나 코드 표시(…) 안에 넣기

■ 확인 문제 (정답은 부록 A)

1. 월별 매출 그래프에서 막대가 한 달씩 밀려 보인다면 무엇을 확인해야 하나요?
2. 주문 금액의 대표값으로 평균보다 중앙값이 나은 이유는?

CHAPTER 5

주요 인사이트 10가지

이 데이터로 찾을 수 있는 핵심 발견 — 근거 · 의미 · 주의점

학습 목표

- Olist 데이터에서 찾을 수 있는 핵심 인사이트 10가지를 숫자와 그림으로 설명한다.
- 인사이트를 '사실 → 근거 → 의미 → 제안' 순서의 문장으로 쓴다.
- 각 인사이트의 한계(주의점)를 함께 말할 수 있다.

좋은 인사이트는 숫자를 읊는 것이 아니라 **'그래서 무엇을 해야 하는가'** 까지 이어집니다. 아래 10가지는 모두 실제 데이터로 계산한 결과이며, 괄호 안에 근거가 있는 노트북 위치를 적었습니다.

인사이트 1. 고객 대부분이 한 번 사고 떠난다

근거: 노트북 01 · B-2

발견

실제 고객 96,096명 중 2번 이상 구매한 고객은 2,997명, 재구매율 3.12% 에 불과하다.

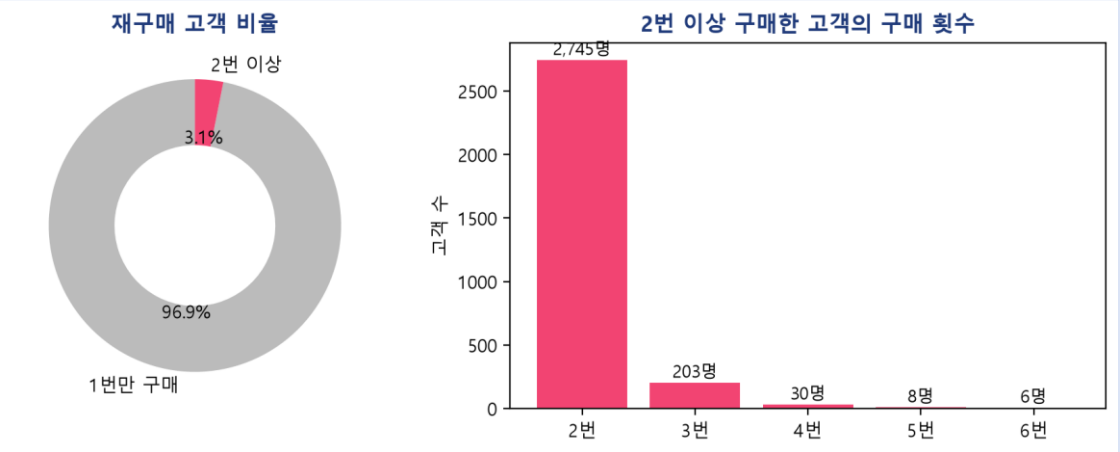


그림 5-1 재구매 고객 비율과 구매 횟수

- **의미 · 제안** — 신규 고객 확보 비용을 생각하면 재구매 유도(첫 구매 후 쿠폰, 재구매 알림, 추천)가 가장 큰 성장 기회다. 인사이트 9(배송 경험)와 연결해 원인을 찾는다.
- **주의** — Olist 는 여러 쇼핑몰에 상품을 연결해 주는 회사라, 같은 고객이 다른 경로로 산 기록은 이 데이터에 없을 수 있다.

인사이트 2. 고객은 남동부 3개 주에 몰려 있다

근거: 노트북 01 · B-3

발견

상파울루(SP) 한 곳에 고객의 약 42%, SP · RJ · MG 3개 주에 약 66.5% 가 있다.

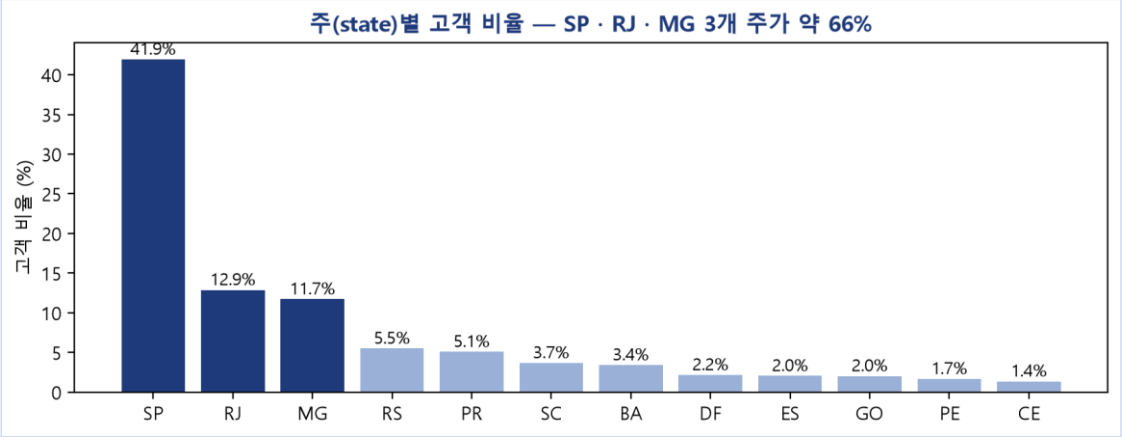


그림 5-2 주(state)별 고객 비율 (상위 12개)

- **의미 · 제안** — 마케팅 · 물류 자원은 남동부에 집중하되, 고객이 적은 북부 · 북동부는 배송 기간(인사이트 10)이 걸림돌인지 확인해 시장 확대 전략을 세운다.
- **주의** — 인구가 많은 지역에 고객이 많은 것은 당연할 수 있다. 인구 대비 고객 비율(침투율)로 다시 보면 다른 결론이 나올 수 있다.

인사이트 3. 신용카드 할부가 구매의 기본이다

근거: 노트북 01 · B-4

발견

주문의 약 75% 가 신용카드, 약 20% 가 볼레투(boleto, 은행 고지서 결제). 신용카드 결제의 약 3분의 2가 할부(일시불 33%, 평균 3.5개월)다.

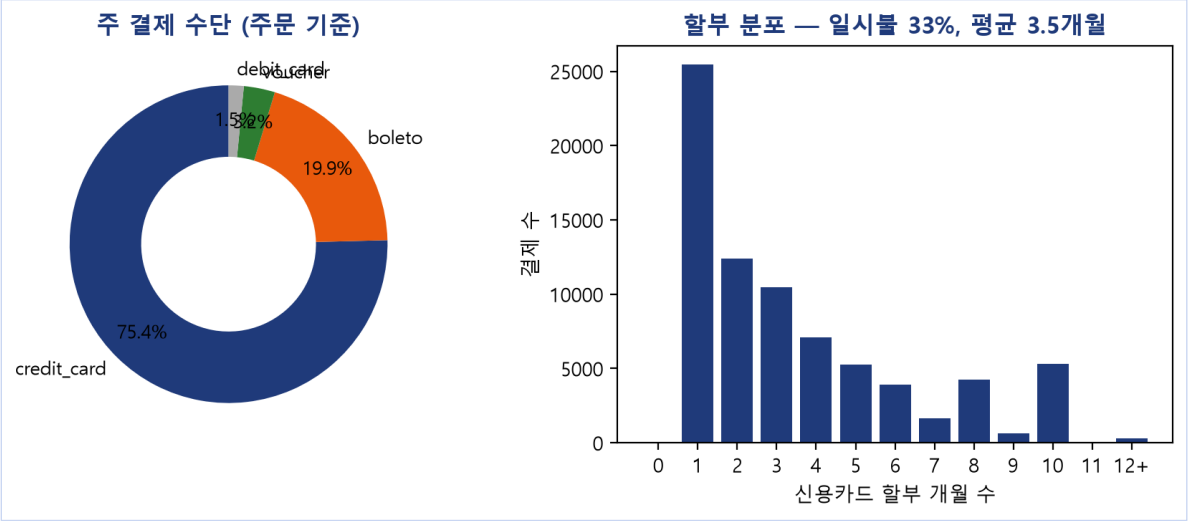


그림 5-3 주 결제 수단 비율과 신용카드 할부 개월 수

- **의미 · 제안** — 무이자 할부 혜택이 구매 결정에 중요한 시장이다. 고가 상품일수록 할부 개월 수가 늘어나는지 추가로 확인해 볼 만하다.
- **주의** — 결제 수단을 2가지 이상 쓴 주문(약 2,200건)은 금액이 가장 큰 수단 하나로 셌다(중복 집계 방지).

인사이트 4. '보통 주문'은 평균보다 훨씬 작다

근거: 노트북 01 · C-3

> **발견**
>
> **주문 1건의 평균 금액(객단가)은 약 160헤알이지만 중앙값은 약 105헤알이다. 주문의 약 80% 가 200헤알 이하다.**

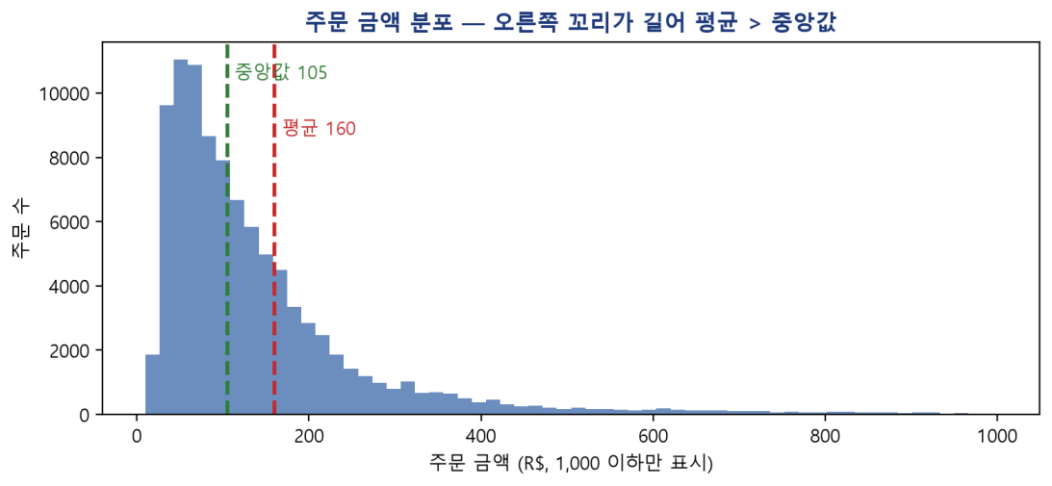


그림 5-4 주문 금액 분포 (1,000헤알 이하)

- **의미 · 제안** — 프로모션 기준 금액(예: 무료 배송 기준)은 평균이 아니라 중앙값 · 분포를 보고 정해야 대부분의 고객에게 의미가 있다.
- **주의** — 평균만 보고하면 '보통 고객'의 모습을 과대평가한다. 금액 데이터는 항상 평균과 중앙값을 함께 쓴다.

인사이트 5. 성장은 객단가가 아니라 주문 수에서 왔다

근거: 노트북 01 · C-6, 노트북 02 · D-3

> **발견**
>
> **같은 1~8월끼리 비교하면 2018년 매출이 2017년보다 143.3% 늘었다(주문 수 +139.9%).**
>
> **그동안 월별 객단가는 146.3~170.1헤알 사이에서 거의 그대로였다.**

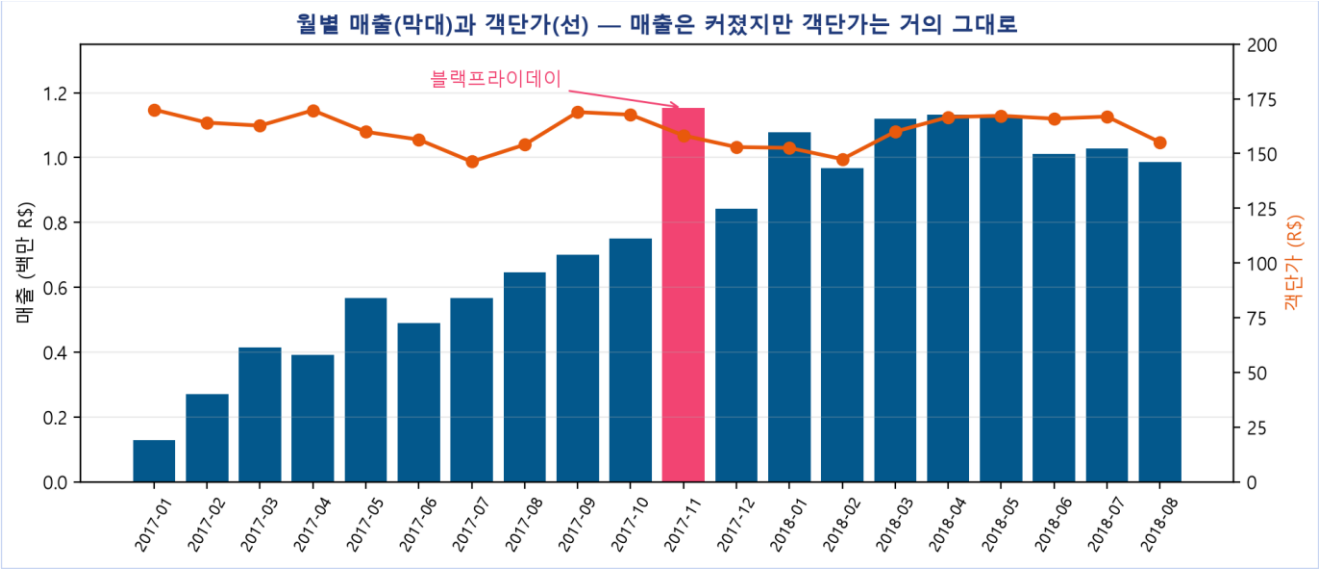


그림 5-5 월별 매출(막대)과 객단가(선), 2017.01~2018.08

- **의미 · 제안** — 회사는 '더 비싸게'가 아니라 '더 많이' 팔아서 성장했다. 신규 고객 유입 채널을 유지하면서 인사이트 1(재구매)을 개선하면 성장이 더 커진다.
- **주의** — 2018년 9월 이후 데이터가 없어 연말 성수기를 포함한 연간 비교는 할 수 없다.

인사이트 6. 블랙프라이데이 — 그리고 공정한 비교의 중요성

근거: 노트북 01 · C-4, 노트북 02 · D-1 · D-4

> **발견**
>
> **2017년 11월 24일(블랙프라이데이) 하루에만 1,100건이 넘는 주문이 들어와 11월이 최대 매출 월이 되었다. 그런데 분기 합계를 모든 연도를 섞어 계산하면 4분기가 '가장 낮은' 분기로 나온다 — 2018년 4분기 데이터가 아예 없기 때문이다.**

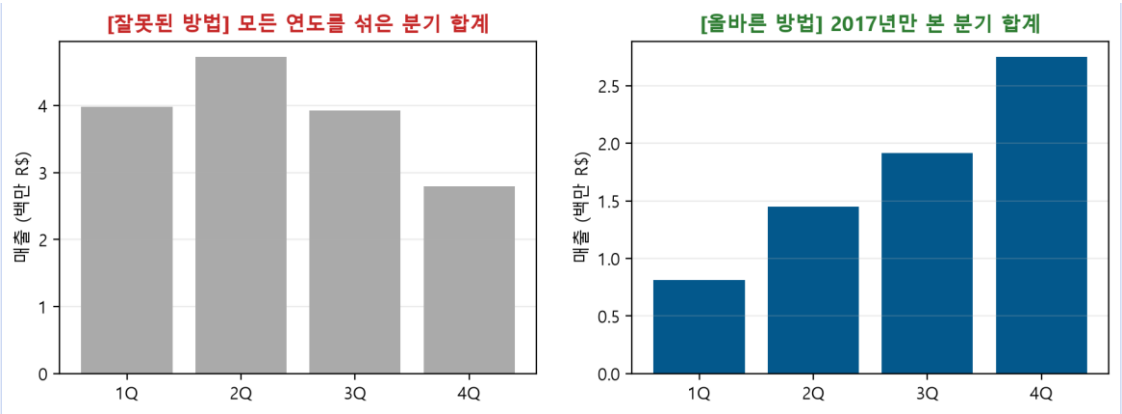


그림 5-6 같은 데이터, 다른 결론 — 비교 기간을 맞추면 4분기가 1위

- **의미 · 제안** — 성수기(블랙프라이데이) 전에 재고 · 물류 인력을 늘려야 한다. 그리고 그룹마다 데이터 기간이 같은지 확인하는 습관이 잘못된 의사 결정을 막는다.
- **주의** — 2017년 한 해의 블랙프라이데이만 관찰했으므로 매년 같은 규모인지는 알 수 없다.

인사이트 7. 평일 낮에 사고, 새벽 · 주말에는 덜 산다

근거: 노트북 02 · D-5 ~ D-7

발견

월요일 주문이 가장 많고 토요일이 가장 적다. 하루 중에는 10~22시에 주문이 몰리고(최다 16시), 새벽 5시가 가장 적다.

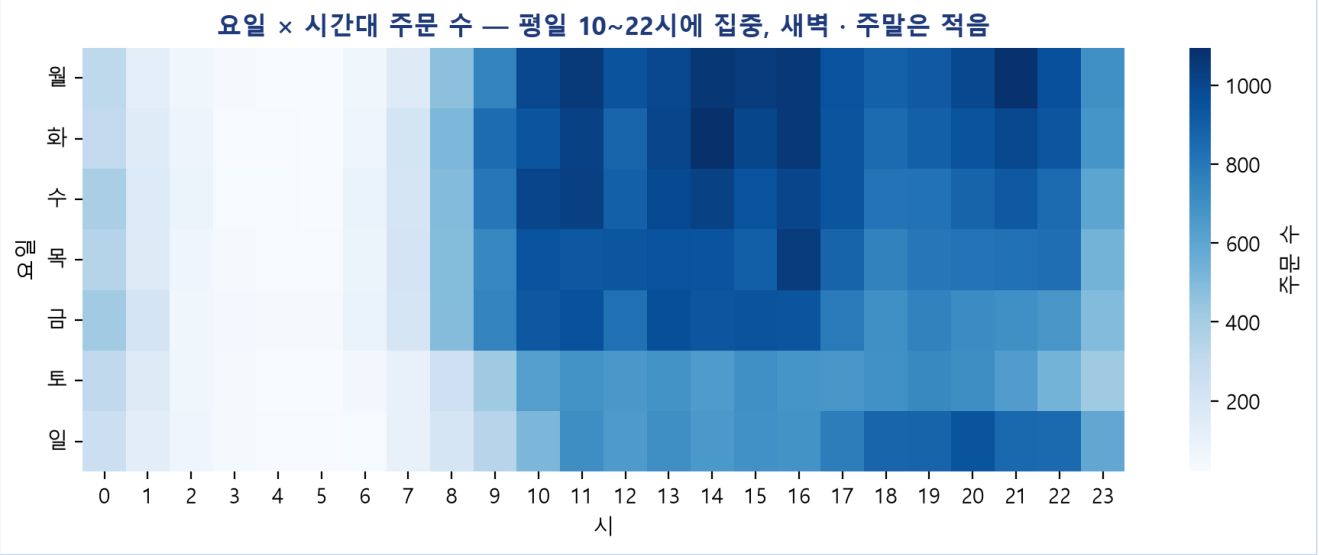


그림 5-7 요일 × 시간대 주문 수 히트맵

- **의미 · 제안** — 푸시 알림 · 타임 세일은 평일 오전 10~11시, 오후 2~4시에 맞추고, 고객센터 인력도 이 시간대에 배치한다.
- **주의** — 이 히트맵은 '조합별 주문 수'이지 두 변수의 상관계수가 아니다. 또 시간은 브라질 현지 시각 기준이다.

인사이트 8. 많이 팔리는 것과 돈이 되는 것은 다르다

근거: 노트북 02 · E-3 · E-4

발견

판매 수량 1위는 bed_bath_table(침구 · 욕실), 매출 1위는 health_beauty(건강 · 미용)이다. watches_gifts 는 수량 7위지만 평균 가격이 높아 매출 2위다. 카테고리 74개 중 상위 10개가 매출의 62.4%, 상위 18개가 80% 를 차지한다.

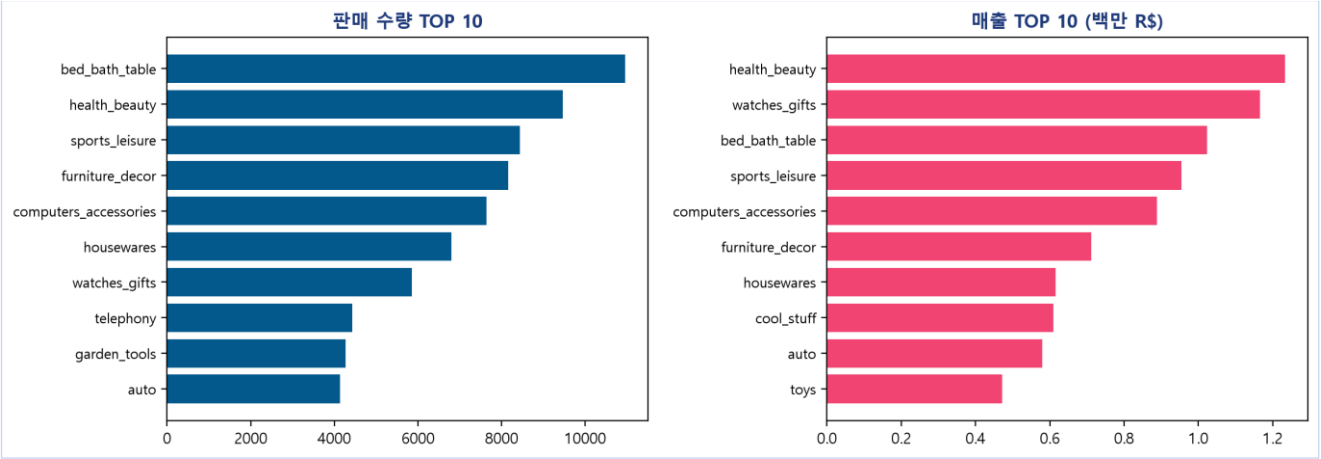


그림 5-8 판매 수량 TOP 10 과 매출 TOP 10

- **의미 · 제안** — 핵심 카테고리(상위 18개)는 재고 · 판매자 관리를 집중하고, 고단가 카테고리는 할부 혜택(인사이트 3)과 묶어 판매한다.
- **주의** — 매출은 상품 가격 합계(배송비 제외)다. 이익(마진) 데이터가 없으므로 '돈이 된다'는 매출 기준의 말이다.

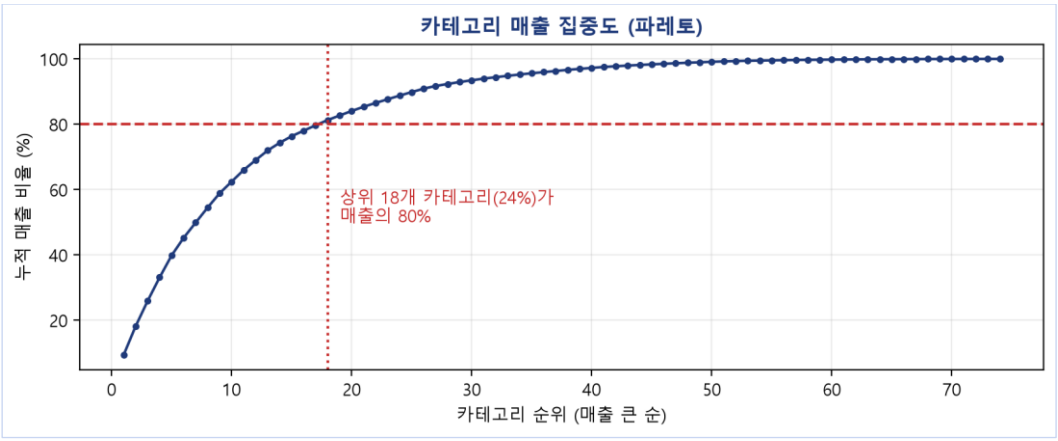


그림 5-9 카테고리 누적 매출 비율 (파레토)

인사이트 9. 배송이 늦으면 리뷰가 무너진다 (가장 중요)

근거: 노트북 02 · F-4 · F-5

> **발견**
>
> **약속한 날짜보다 늦게 도착한 주문(전체의 8.1%)의 평균 리뷰는 2.55점으로, 제때 도착한 주문(4.28점)보다 1.7점 낮다. 지연 주문의 46.6% 가 1점(제때 도착은 6.8%). 배송이 3주를 넘으면 점수가 급격히 떨어진다.**

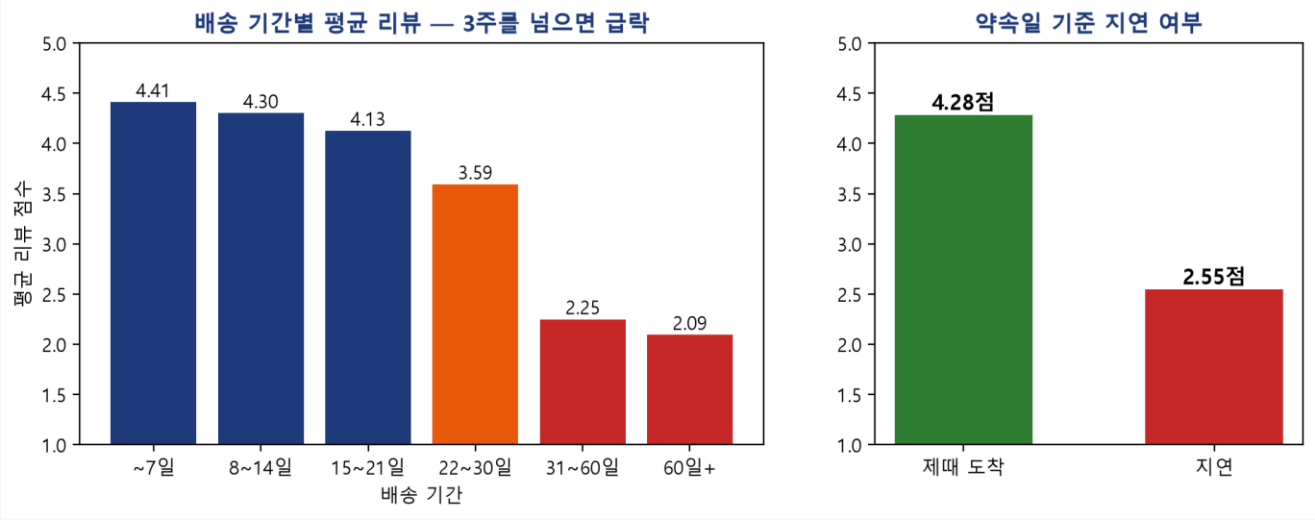


그림 5-10 배송 기간별 · 지연 여부별 평균 리뷰 점수

- **의미 · 제안** — 고객 만족의 가장 큰 열쇠는 '빠른 배송'보다 '약속을 지키는 배송'이다. 지연 위험이 큰 주문에 미리 안내 · 보상을 하면 낮은 리뷰와 이탈(인사이트 1)을 줄일 수 있다.
- **주의** — 관계(상관)가 강하지만, 지연이 리뷰를 낮춘 '원인'이라고 단정하려면 상품 · 판매자 등 다른 요인을 통제한 추가 분석이 필요하다.

인사이트 10. 주문이 몰리면 지연이 늘고, 먼 지역은 더 오래 걸린다

근거: 노트북 02 · F-3 · F-4 · F-6

발견

배송 기간 중앙값은 10.2일(약속 기간 중앙값 23.2일)이다. 평소 월별 지연율은 대체로 3~8% 수준이지만 블랙프라이데이(2017.11)에는 14%, 2018년 3월에는 21% 까지 치솟았다. 상파울루(SP) 고객은 약 7.2일, 북부(AM · RR · AP)는 약 25일이 걸린다.

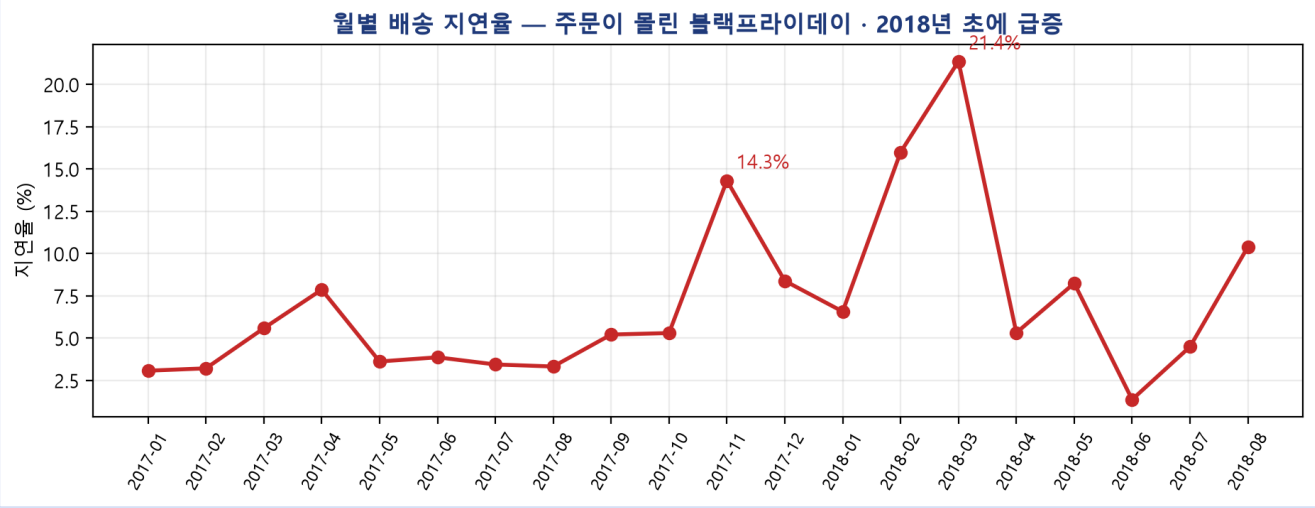


그림 5-11 월별 배송 지연율

- **의미 · 제안** — 성수기에는 약속 기간을 늘리거나 물류를 증원하고, 먼 지역에는 현실적인 예정일을 안내하거나 지역 물류 거점을 검토한다.
- **주의** — 2018년 하반기 배송이 빨라진 원인(물류 개선? 지역 · 상품 구성 변화?)은 이 데이터만으로 알 수 없다. 마지막 달들은 아직 배송 중인 주문이 빠져 실제보다 빨라 보일 수 있다.

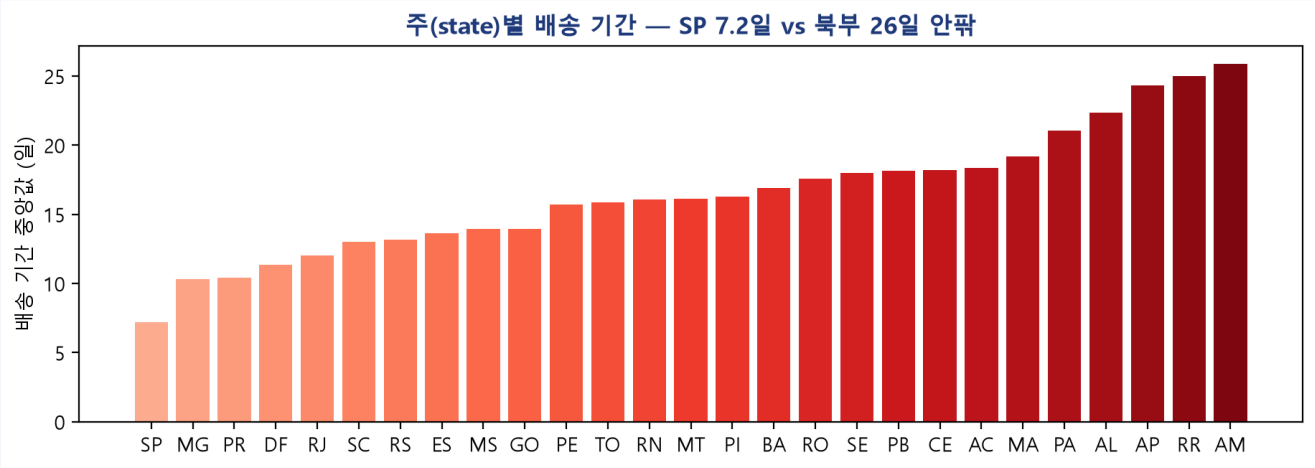


그림 5-12 주(state)별 배송 기간 중앙값

5.11 인사이트를 실행으로 — 한 장 요약

번호	인사이트	제안하는 행동	측정할 지표(KPI)
1	재구매율 약 3%	첫 구매 후 재구매 쿠폰 · 알림	재구매율, 90일 내 재구매 고객 수
2	남동부 3개 주에 66%	지역별 마케팅 · 물류 배분	주별 고객 수 · 침투율
3	카드 할부가 기본	무이자 할부 프로모션	할부 결제 비율, 고가 상품 전환율
4	평균 160 · 중앙값 105	프로모션 기준 금액을 분포로 설정	중앙값 객단가
5	성장 = 주문 수	유입 채널 유지 + 재구매 개선	월 주문 수, 신규 고객 수
6	블랙프라이데이 폭증	성수기 재고 · 물류 사전 증원	성수기 일 주문 수 · 지연율
7	평일 낮 집중	알림 · 타임 세일 · 상담 인력 시간대 조정	시간대별 전환율
8	수량 ≠ 매출, 상위 18개가 80%	핵심 카테고리 집중 관리	카테고리별 매출 · 점유율
9	지연 시 리뷰 4.3 → 2.6	지연 위험 주문 사전 안내 · 보상	지연율, 평균 리뷰, 1점 비율
10	성수기 · 원거리 지연	예정일 현실화 · 지역 거점	월별 · 주별 지연율, 배송 중앙값

도전 과제 — 인사이트 문장 쓰기

- 형식: **[사실]** 숫자로 무엇을 발견했나 → **[근거]** 어떤 데이터 · 기간 · 기준으로 → **[의미]** 그래서 무엇이 중요한가 → **[제안]** 무엇을 하자
- 예) [사실] 지연 주문의 평균 리뷰는 2.55점으로 제때 온 주문(4.28점)보다 1.7점 낮다. [근거] 2016~2018년 배송 완료 주문 약 9.6만 건, 약속일 기준. [의미] 배송 약속이 만족도를 좌우한다. [제안] 성수기 예정일을 현실화하고 지연 위험 주문에 미리 안내한다.
- 직접 해 보기: 대시보드의 필터로 특정 주(예: RJ)만 골라 위 형식의 인사이트 문장 2개를 써 보세요.

■ 확인 문제 (정답은 부록 A)

1. 인사이트 6에서 분기 합계를 모든 연도로 섞어 계산하면 4분기가 낮게 나오는 이유는?
2. 인사이트 9(배송 지연과 리뷰)의 주의점은?
3. 인사이트를 문장으로 쓸 때의 4단계 순서는?

CHAPTER 6

Streamlit 대시보드 이해하기

노트북 03 보충 — app.py 해부와 내 컴퓨터에서 실행하기

학습 목표

- Streamlit 이 화면을 그리는 원리(rerun)와 캐시를 이해한다.
- app.py 의 구조를 6개 구역으로 나눠 설명할 수 있다.
- 대시보드를 내 컴퓨터에서 실행하고 직접 고쳐 본다.

6.1 Streamlit 의 원리 — 조작할 때마다 다시 실행

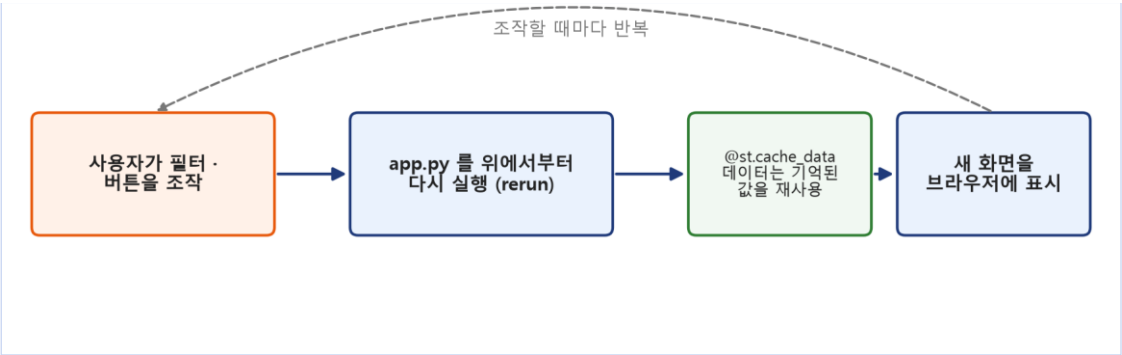


그림 6-1 Streamlit 의 실행 방식

Streamlit 은 파이썬 코드만으로 웹 화면을 만드는 라이브러리입니다. 사용자가 필터를 바꾸면 **app.py 전체를 위에서부터 다시 실행**해 새 화면을 그립니다. 그래서 데이터 읽기처럼 오래 걸리는 작업은 @st.cache_data 로 결과를 기억해 두어야 빠릅니다.

6.2 app.py 의 구조

구역	하는 일	주요 코드
① 페이지 설정	브라우저 탭 제목 · 아이콘 · 넓은 화면	st.set_page_config(page_title=..., layout="wide")
② 데이터 불러오기	app.py 기준 data 폴더에서 CSV 읽기 + 캐시	DATA_DIR = Path(__file__).parent / "data" @st.cache_data
③ 사이드바 필터	주문 기간 · 고객 지역 선택 → 조건에 맞는 행만 남김	st.sidebar.date_input · multiselect mask = ...between(start, end)
④ 제목 · KPI	핵심 지표 6개를 3개씩 두 줄로	st.columns(3) + st.metric
⑤ 탭 6개	매출 · 주문 시간 · 카테고리 · 배송리뷰 · 지역 · 데이터	st.tabs([...]) + with tab: ... st.plotly_chart
⑥ 출처 표시	데이터 출처 · 라이선스	st.caption("데이터: Olist ...")

[코드] app.py 의 핵심 부분

```
@st.cache_data          # 한 번 읽은 결과를 기억 → 필터를 바꿀 때마다 다시 읽지 않음
def load_data():
    orders = pd.read_csv(DATA_DIR / "orders.csv.gz", parse_dates=["purchase_ts"])
    items = pd.read_csv(DATA_DIR / "items.csv.gz")
    return orders, items


period = st.sidebar.date_input("주문 기간", value=(first_day, last_day))
if len(period) != 2:            # 달력에서 시작일만 고른 순간
    st.info("끝 날짜까지 선택해 주세요."); st.stop()


k1, k2, k3 = st.columns(3)      # 한 줄에 6개를 넣으면 숫자가 잘림 → 3개씩 두 줄
k1.metric("주문 수", f"{len(f_orders):,}건")
```

경로는 app.py 기준으로

pd.read_csv("data/orders.csv.gz") 처럼 쓰면 **실행한 위치** 기준으로 파일을 찾아서, 다른 폴더에서 실행하거나 배포 서버에서 FileNotFoundError 가 날 수 있습니다. **Path(__file__).parent / "data"** 는 'app.py 가 있는 폴더의 data' 를 뜻하므로 어디서 실행해도 맞습니다.

6.3 탭별 화면



그림 6-2 주문 시간 탭 — 요일 × 시간대 히트맵 (인사이트 7)



그림 6-3 카테고리 탭 — 기준(매출/판매 수량)과 개수를 바꿔 볼 수 있다 (인사이트 8)

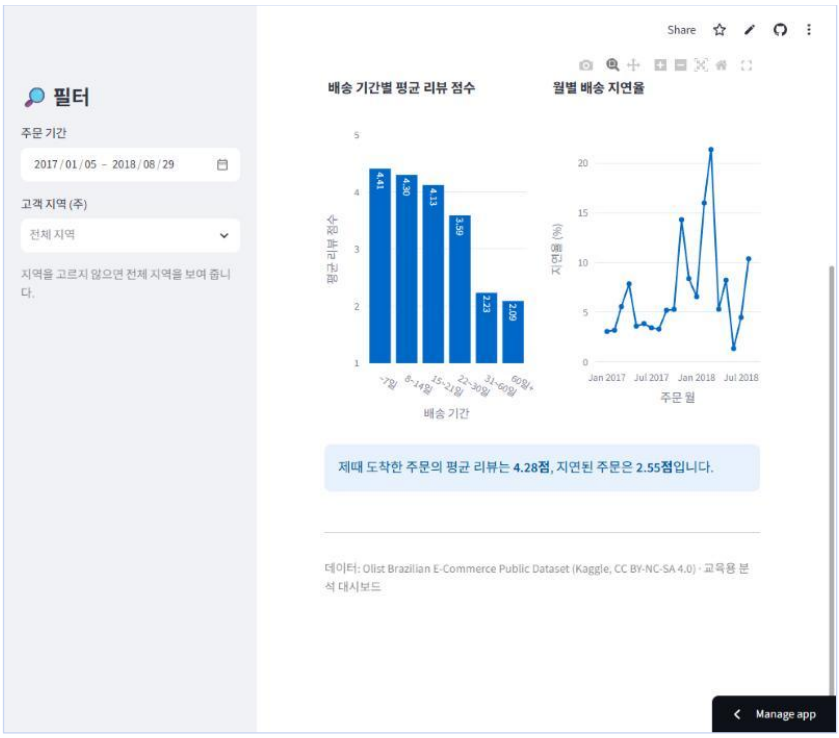


그림 6-4 배송 · 리뷰 탭 — 배송 기간별 리뷰와 월별 지연율 (인사이트 9 · 10)

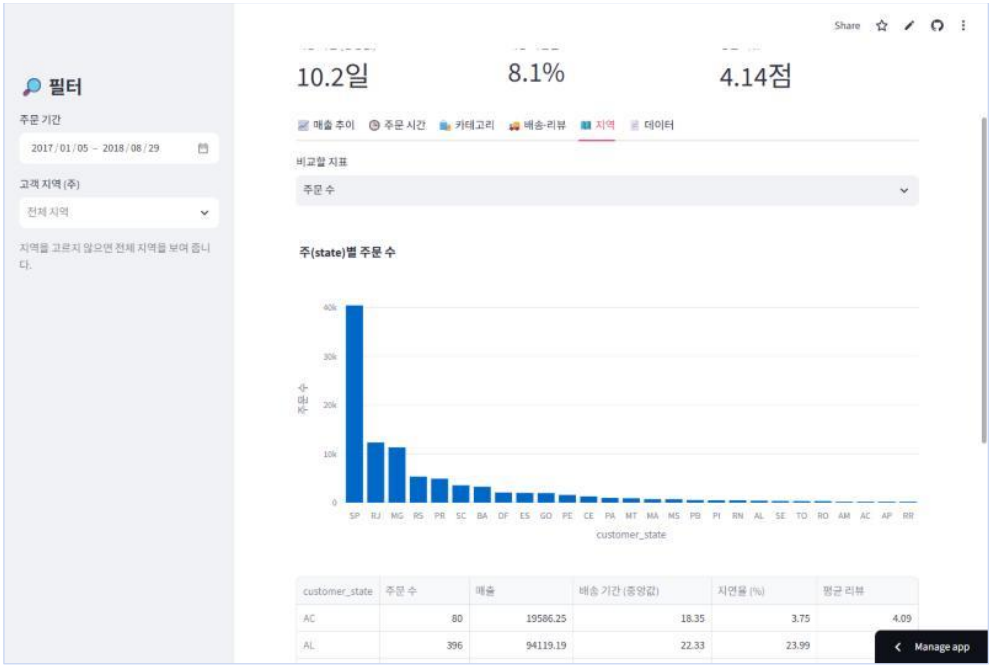


그림 6-5 지역 탭 — 비교할 지표를 골라 주(state)별로 비교

6.4 내 컴퓨터에서 실행하기

1. 03 노트북을 위에서부터 끝까지 실행합니다. olist_dashboard 폴더와 그 안의 파일들이 만들어집니다(5-1 절 자동 테스트에서 '오류: 없음 ✓' 확인).
2. VS Code 터미널(저장소 최상위에서 열림)에서 notebooks/olist_dashboard 폴더로 이동한 뒤 streamlit run 을 실행합니다.
3. 브라우저에 http://localhost:8501 이 열리면 성공입니다. 필터와 탭을 눌러 봅니다.
4. 끝낼 때는 터미널을 클릭하고 **Ctrl + C** 를 누릅니다.

[터미널]

```
cd notebooks/olist_dashboard
streamlit run app.py
```

도전 과제 — 직접 고쳐 보기

- app.py 의 st.title(...) 문구를 '내 이름의 Olist 대시보드'로 바꾸고 저장 → 브라우저 오른쪽 위 **Rerun** 을 눌러 반영되는지 확인
- 결제 수단 필터 추가: 지역 필터 코드를 참고해 st.sidebar.multiselect("결제 수단", ...) 를 만들고 mask 에 orders["payment_type"].isin(...) 조건을 더해 보세요.
- 고친 내용은 8.9 절 방법으로 push 하면 배포된 사이트에도 반영됩니다.

■ 확인 문제 (정답은 부록 A)

1. @st.cache_data 를 쓰지 않으면 어떤 문제가 생기나요?
2. 데이터 경로에 Path(__file__).parent 를 쓰는 이유는?

CHAPTER 7

GitHub 에 올리기

git 으로 기록하고 GitHub 저장소에 push — 처음부터 끝까지

학습 목표

- git 의 작업 흐름(add → commit → push)을 설명할 수 있다.
- GitHub 에 저장소를 만들고 olist_dashboard 폴더를 올린다.

7.1 git 과 GitHub

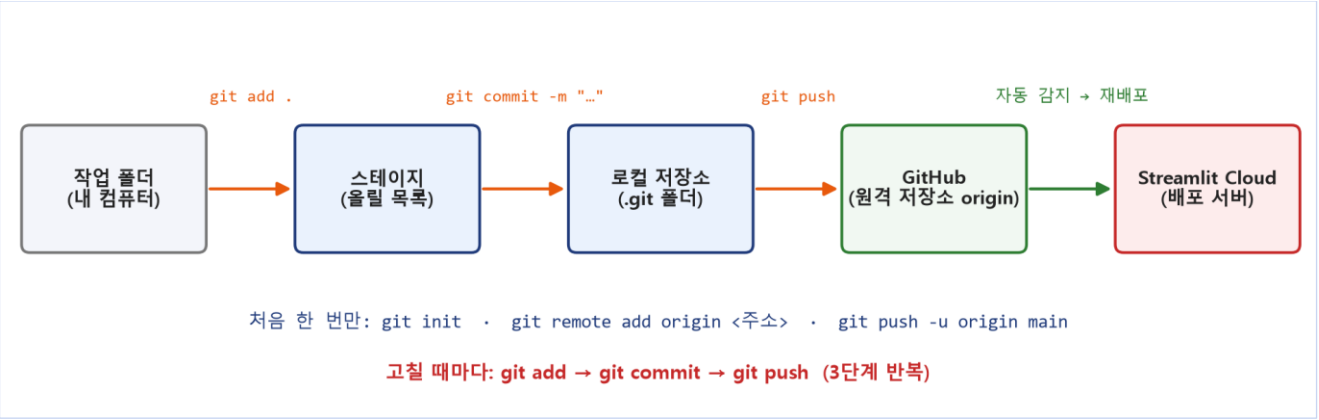


그림 7-1 git 의 작업 흐름 — 고칠 때마다 add → commit → push 3단계 반복

용어	뜻
git	내 컴퓨터에서 파일의 변경 이력을 기록하는 프로그램
저장소 (repository)	git 이 관리하는 프로젝트 폴더 (안에 숨김 폴더 .git 이 생김)
commit	현재 상태를 메시지와 함께 기록 ('저장하기')
GitHub	저장소를 인터넷에 올려 두는 서비스 ('클라우드 저장')
remote · origin	내 저장소와 연결된 GitHub 저장소 주소 (기본 이름이 origin)
push	내 commit 들을 GitHub 로 올리기

7.2 (처음 한 번) GitHub 가입과 git 설정

1. https://github.com 에서 **Sign up** 으로 가입합니다(구글 계정으로도 가능). 아이디(예: sinaboro)가 저장소 주소에 들어갑니다.
2. 터미널에서 git 이 설치되어 있는지 확인합니다. 'git version 2.xx' 가 나오면 됩니다.
3. commit 에 기록될 이름과 이메일을 등록합니다. 이메일은 **GitHub 가입 이메일**을 쓰세요.

[터미널]

```
git --version
git config --global user.name "내 이름"
git config --global user.email "github가입이메일@example.com"
```

7.3 GitHub 에 빈 저장소 만들기

1. GitHub 오른쪽 위 **+** → **New repository** 를 누릅니다.
2. **Repository name** 에 저장소 이름을 씁니다(영문 · 숫자 · 하이픈, 예: olist-dashboard).
3. **Choose visibility** 는 **Public** (Streamlit 무료 배포는 공개 저장소가 가장 간단합니다).
4. **Add README** 는 **Off** 그대로 둡니다. 켜면 첫 push 가 거절될 수 있습니다(9장).
5. 초록 **Create repository** 를 누르고, 화면에 나오는 주소 https://github.com/내아이디/저장소이름.git 을 복사합니다.

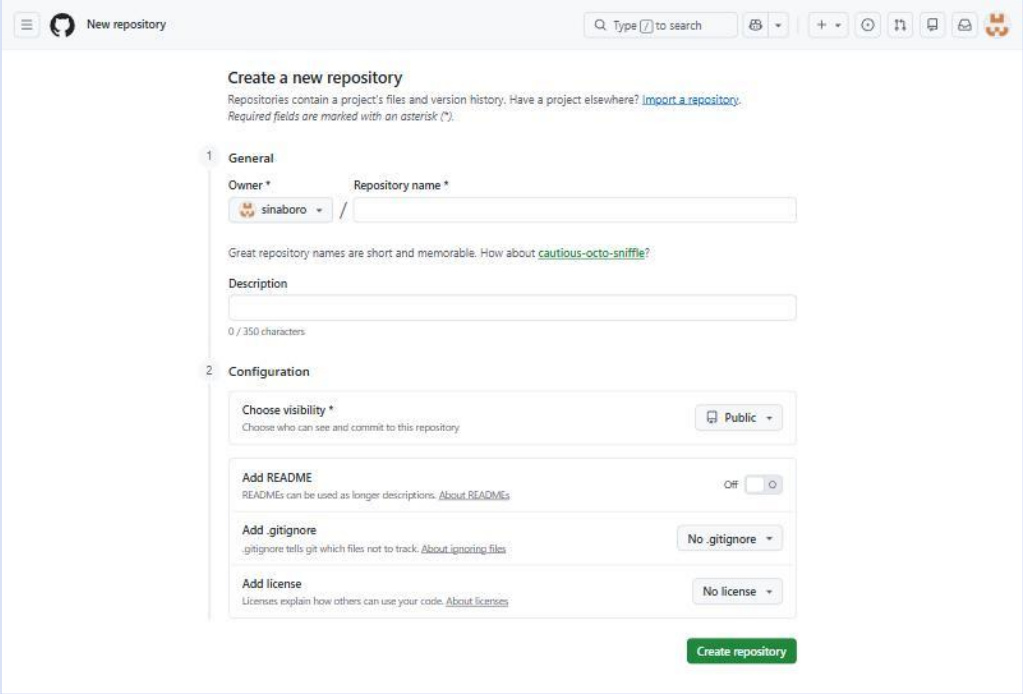


그림 7-2 GitHub 새 저장소 만들기 화면 — 이름 입력, Public, README Off

7.4 내 컴퓨터에서 commit 하고 push 하기

내 GitHub 에 새로 만드는 저장소이므로 반드시 **notebooks/olist_dashboard 폴더 안에서** 실행합니다. 받아 둔 예시 저장소 폴더(최상위)는 이미 예시 GitHub 와 연결되어 있으니 그곳에서 git init 하지 마세요.

[터미널]

```
cd notebooks/olist_dashboard
git init                                  # 이 폴더를 git 저장소로 (처음 한 번)
git add .                                 # 모든 파일을 올릴 목록에 (.gitignore 에 적힌 것은 제외)
git status                                # 올라갈 파일 확인: app.py, data/…, requirements.txt …
git commit -m "Olist 대시보드 첫 버전"
git branch -M main                        # 브랜치 이름을 main 으로
```

```
git remote add origin https://github.com/내아이디/저장소이름.git
git push -u origin main
```

- 처음 push 하면 **GitHub 로그인 창**(Git Credential Manager)이 뜹니다. **Sign in with your browser** 를 눌러 브라우저에서 로그인 · 허용하면 됩니다. 한 번 로그인하면 컴퓨터가 기억합니다.
- 'warning: LF will be replaced by CRLF' 는 Windows 줄바꿈 알림일 뿐 오류가 아닙니다.
- GitHub 저장소 페이지를 새로 고쳐 파일 목록과 README 가 보이면 성공입니다.

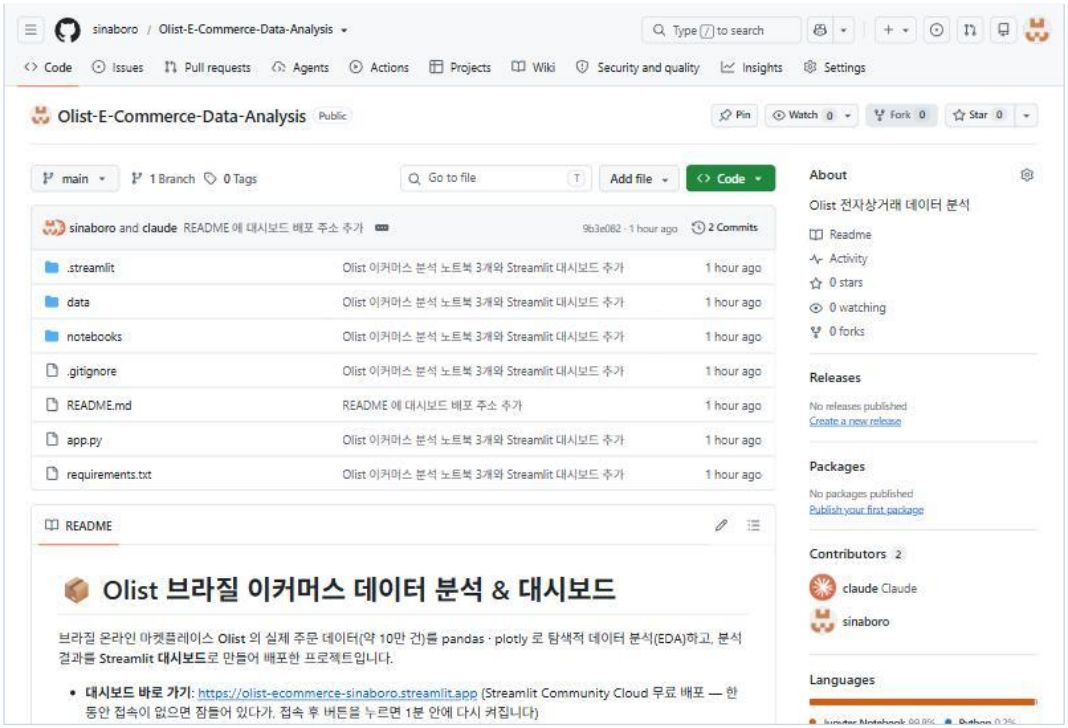


그림 7-3 push 가 끝난 저장소 — 파일 목록 아래에 README.md 내용이 표시된다

올리면 안 되는 것

.gitignore 가 가상환경(venv/), 임시 파일(__pycache__/), **비밀 정보(.streamlit/secrets.toml)** 를 막아 줍니다. git status 결과에 이런 파일이 보이면 commit 하지 마세요. 한 번 올라간 비밀번호 · API 키는 지워도 기록에 남습니다.

더 알아보기 — 노트북도 함께 올리고 싶다면

예시 저장소(https://github.com/sinaboro/Olist-E-Commerce-Data-Analysis)처럼 저장소 최상위에 app.py · data 를 두고 notebooks 폴더에 분석 노트북 3개를 함께 올릴 수도 있습니다. 원본 데이터가 크다면 예시 저장소처럼 .csv.gz 로 압축해 data 폴더에 함께 두고, 압축을 푼 .csv 는 .gitignore 의 data/*.csv 규칙으로 제외하면 저장소를 가볍게 유지할 수 있습니다.

■ 확인 문제 (정답은 부록 A)

1. 고칠 때마다 반복하는 git 명령 3단계는?

2. 새 저장소를 만들 때 Add README 를 켜면 어떤 문제가 생길 수 있나요?

3. git init 을 olist_dashboard 폴더 안에서 해야 하는 이유는?

CHAPTER 8

Streamlit Cloud 로 배포하기

실제 화면으로 따라 하는 무료 배포 — 내 대시보드 주소 만들기

학습 목표

- Streamlit Community Cloud 에 가입하고 GitHub 를 연결한다.
- 저장소 · 브랜치 · 실행 파일 · 주소 · Python 버전을 설정해 배포한다.
- 배포된 앱을 관리(재시작 · 로그 확인 · 업데이트)한다.

8.1 접속하고 로그인하기

https://share.streamlit.io 에 접속해 **Continue to sign-in** 을 누릅니다. **GitHub 계정으로 로그인(Continue with GitHub)** 하는 것이 가장 간단합니다. 구글 계정으로 로그인해도 되지만, 이 경우 8.3 절에서 GitHub 를 따로 연결합니다.

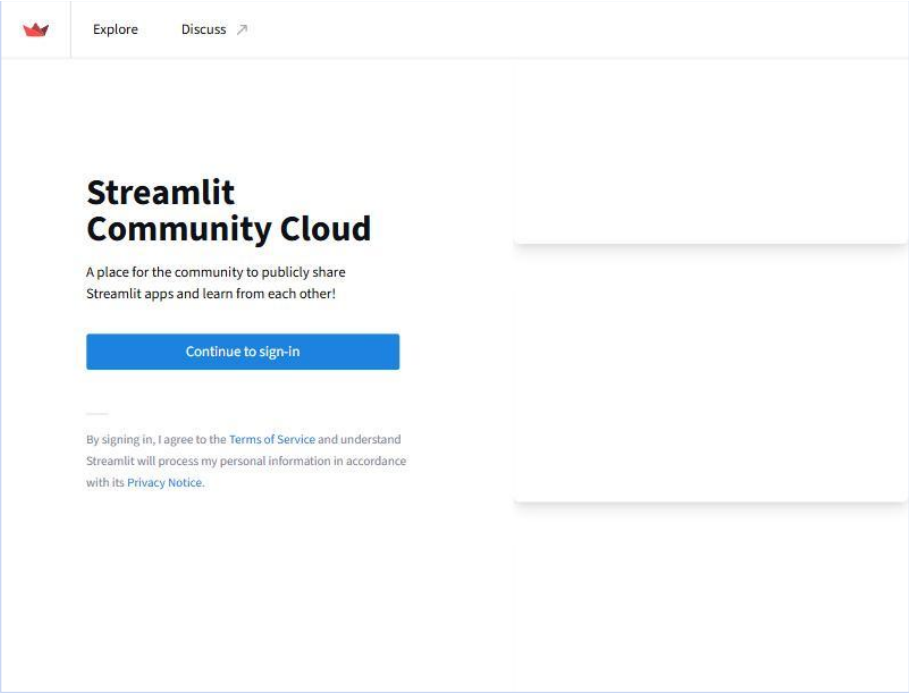


그림 8-1 Streamlit Community Cloud 첫 화면

8.2 (처음 한 번) 계정 정보 입력

처음 로그인하면 **Set up your account** 화면이 나옵니다. 이름 · 이메일은 자동으로 채워져 있고, 아래 세 칸만 고른 뒤 **Continue** 를 누릅니다. 이 화면에서 이용 약관에 동의하게 되므로 본인이 직접 입력합니다.

입력 칸	고를 값 (예시)
What's your functional area? (하는 일)	Education 또는 Data Science 등

입력 칸	고를 값 (예시)
What stage of app development are you at? (개발 단계)	아무거나 (예: Just exploring)
Country or region (국가)	South Korea

8.3 GitHub 연결하기

1. 로그인 후 화면 오른쪽 위 **Create app** 을 누릅니다.
2. GitHub 가 연결되지 않았다면 '**You must be connected to GitHub to deploy an app**' 경고가 뜹니다. **Connect to GitHub** 를 누릅니다.
3. GitHub 로그인 화면이 나오면 로그인합니다(구글로 GitHub 에 가입했다면 **Continue with Google**).
4. **Authorize streamlit** (Streamlit 에 권한 허용) 화면에서 초록 버튼을 눌러 허용합니다. Streamlit 이 내 저장소를 읽어 앱을 실행할 수 있게 해 주는 권한입니다.
5. Streamlit 화면으로 돌아오면 왼쪽 위 작업 공간(Workspace) 이름이 내 GitHub 아이디로 바뀝니다.

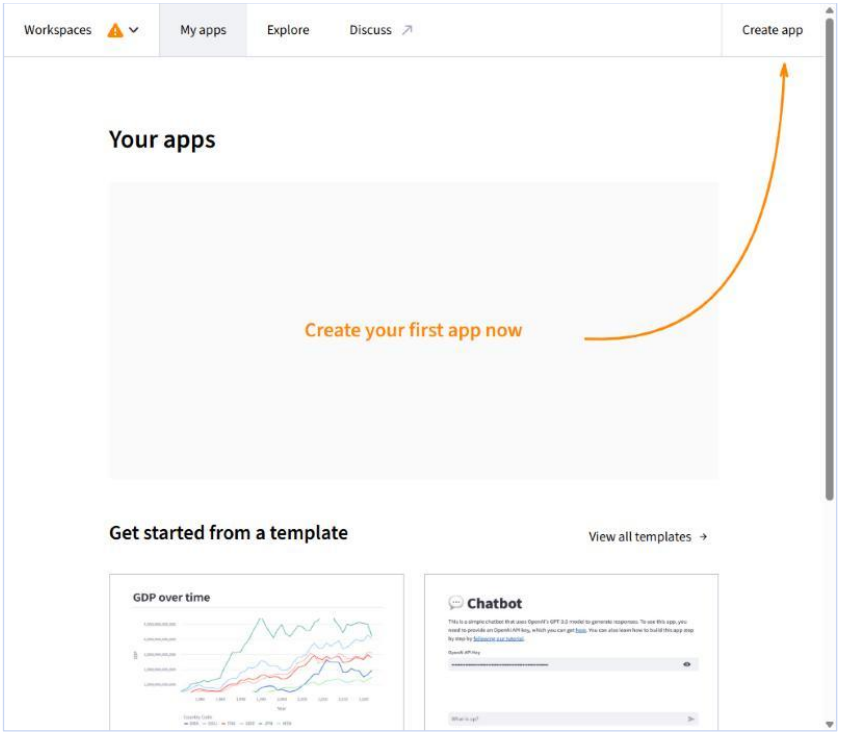


그림 8-2 첫 앱 만들기 — 오른쪽 위 Create app

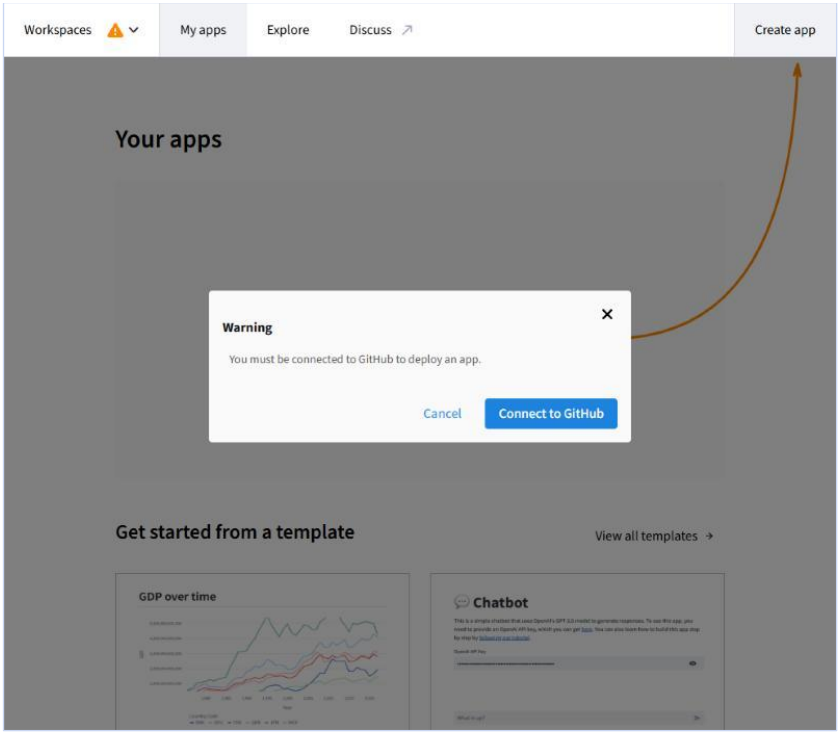


그림 8-3 GitHub 연결이 필요하다는 경고 → Connect to GitHub

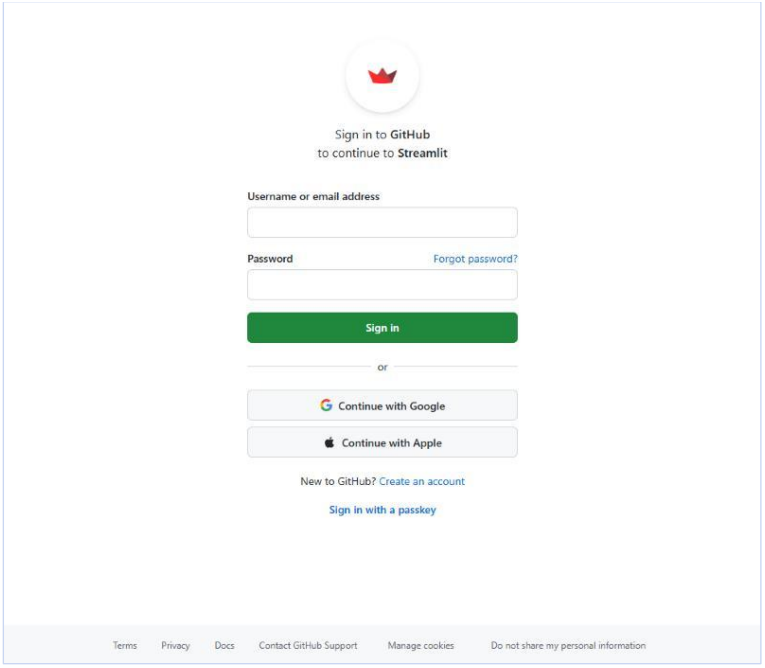


그림 8-4 GitHub 로그인 화면 (구글로 가입했다면 Continue with Google)

8.4 앱 만들기 — 저장소 · 브랜치 · 파일 · 주소

Create app 을 다시 누르면 '**What would you like to do?**' 화면이 나옵니다. 첫 번째 **Deploy a public app from GitHub** 의 **Deploy now** 를 누릅니다.

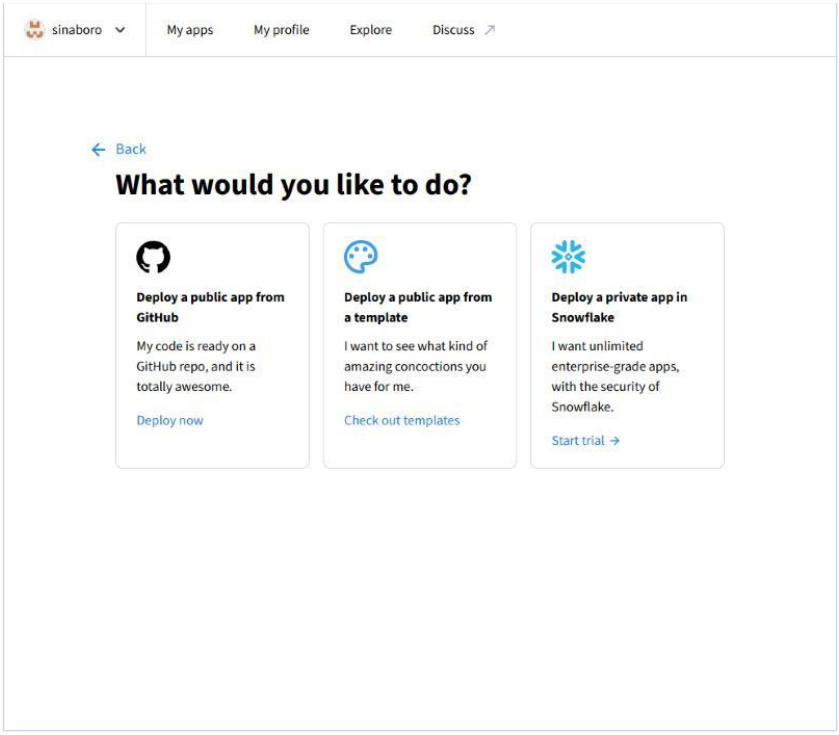


그림 8-5 배포 방식 고르기 — Deploy a public app from GitHub

입력 칸	넣을 값	설명
Repository	내아이디/저장소이름	입력하면 아래에 내 저장소가 목록으로 뜨므로 클릭해서 고름 (오른쪽 'Paste GitHub URL' 로 주소를 붙여 넣어도 됨)
Branch	main	저장소를 고르면 자동으로 채워짐
Main file path	app.py	기본값 streamlit_app.py 를 지우고 app.py 로 (저장소 최상위에 app.py 가 있으므로)
App URL (optional)	원하는 이름 (예: olist-ecommerce-내이름)	아래에 초록색 'Domain is available' 이 나오면 사용 가능. 이 이름이 서비스 주소가 됨

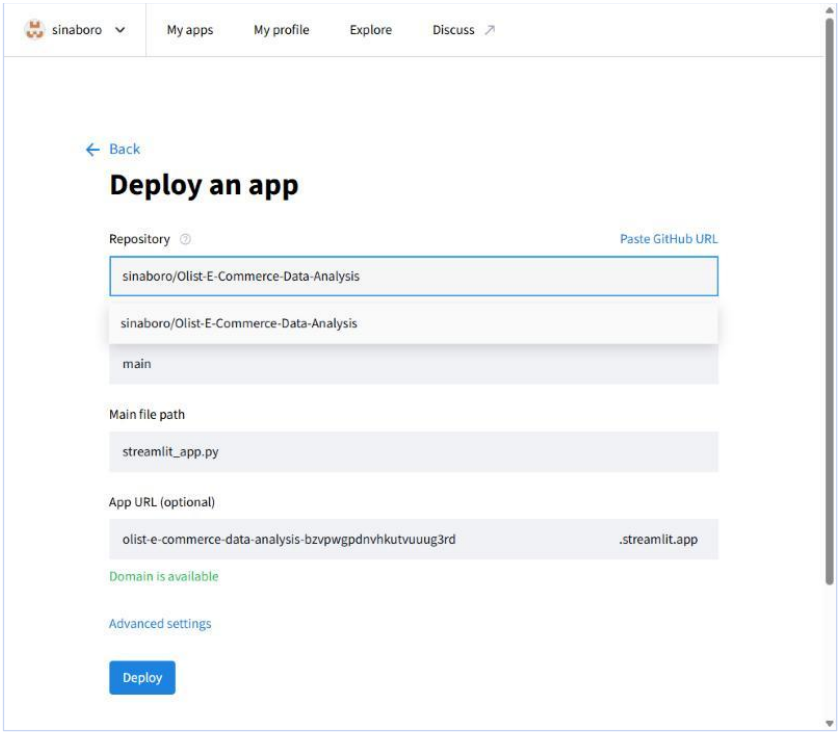


그림 8-6 저장소 이름을 입력하면 내 저장소가 목록에 뜬다

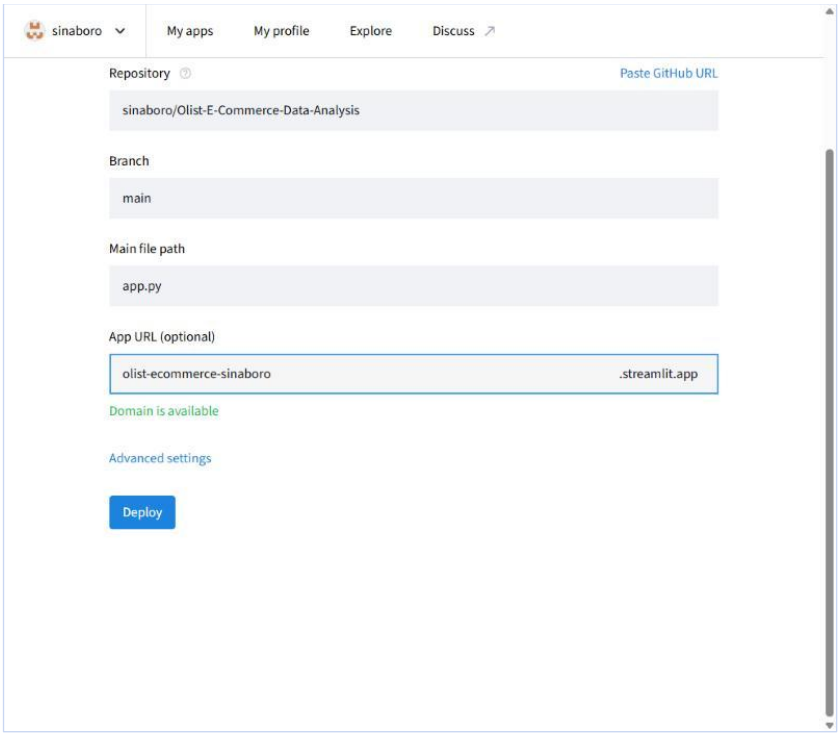


그림 8-7 입력을 마친 화면 — Domain is available 확인

8.5 Python 버전 맞추기 (Advanced settings)

Advanced settings 를 눌러 **Python version** 을 내 컴퓨터와 같은 버전(이 교안은 **3.12**)으로 고르고 **Save** 를 누릅니다. 기본값은 가장 최신 버전이라, 일부 패키지가 아직 지원하지 않으면 설치 오류가 날 수 있습니다. 아래 **Secrets** 칸은 API 키 같은 비밀 정보를 넣는 곳으로, 이 대시보드에는 필요 없습니다.

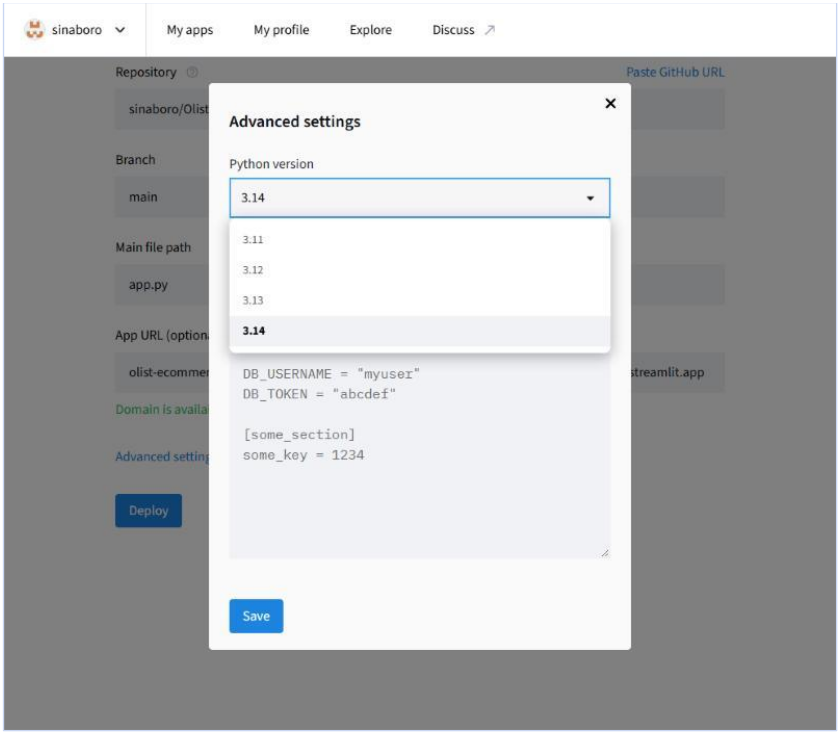


그림 8-8 Advanced settings — Python 버전 선택

8.6 Deploy — 서비스 시작

Deploy 를 누르면 서버가 requirements.txt 의 패키지를 설치하고 app.py 를 실행합니다. 1~3분 뒤 대시보드가 나타나면 주소창의 **https://내가정한이름.streamlit.app** 이 바로 서비스 주소입니다. 친구의 휴대폰에서도 열리는지 확인해 보세요.

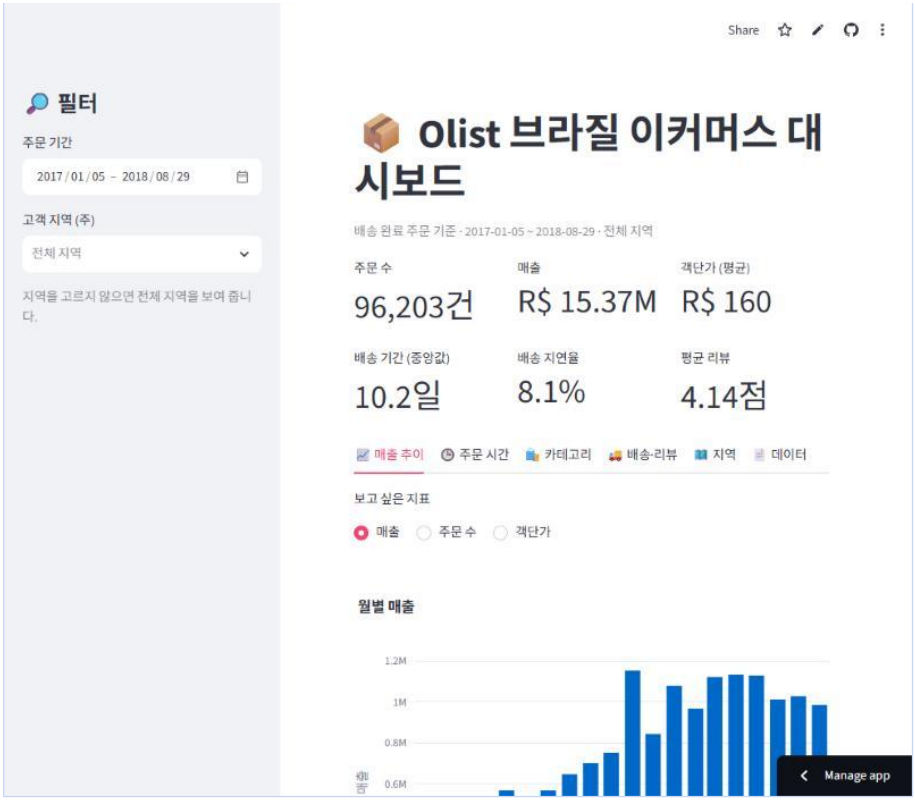


그림 8-9 배포 완료 — 인터넷 주소로 누구나 접속할 수 있다

8.7 앱 관리하기

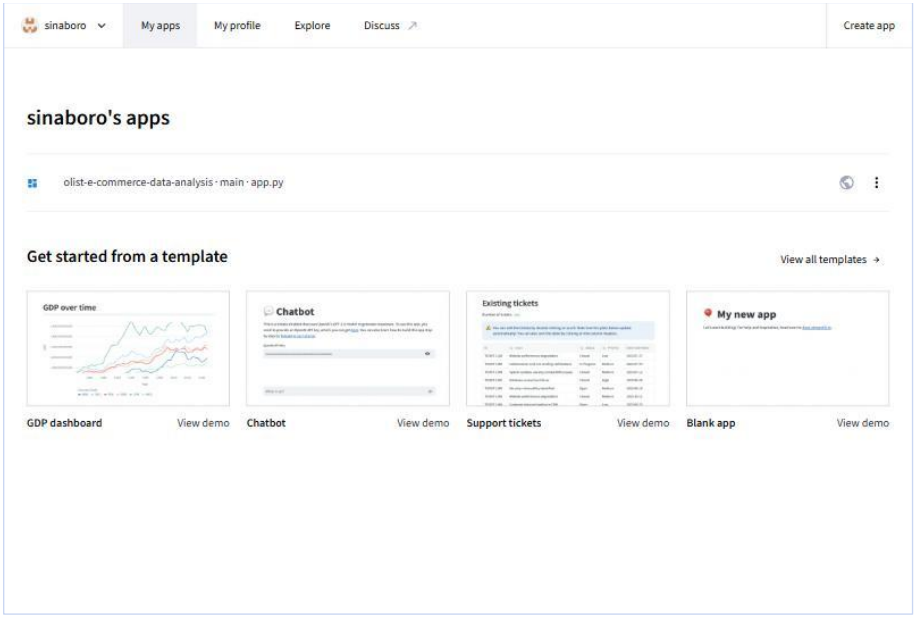


그림 8-10 My apps — 내가 배포한 앱 목록

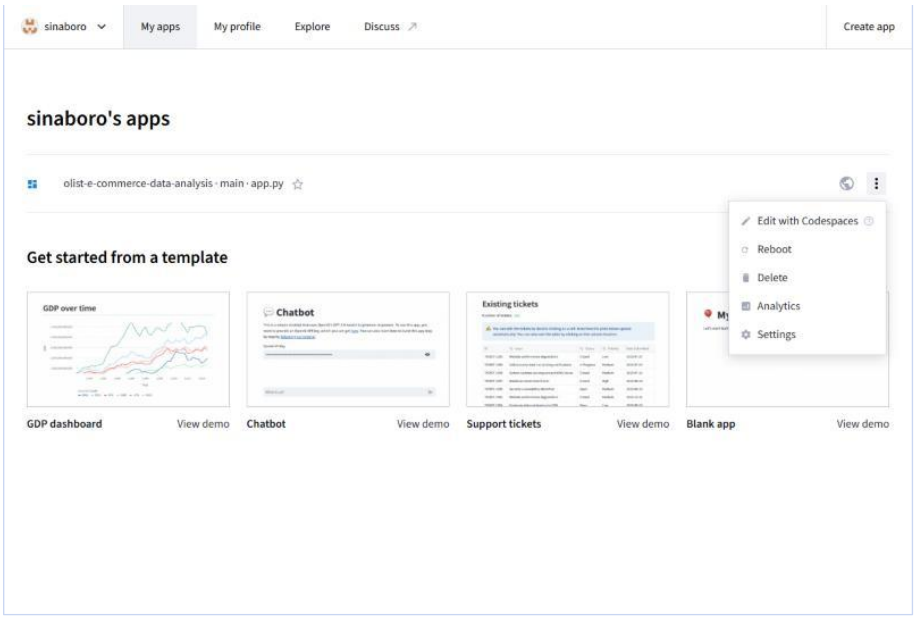


그림 8-11 앱 오른쪽 ⋮ 메뉴 — Reboot(재시작) · Delete(삭제) · Analytics(방문 통계) · Settings(설정)

- **Manage app** — 대시보드 화면 오른쪽 아래 버튼. 누르면 서버 **로그**(패키지 설치 과정, 오류 메시지)가 보입니다. 오류가 나면 가장 먼저 여기를 봅니다.
- **Reboot** — 앱이 멈췄거나 데이터 파일을 바꿨는데 반영이 안 될 때 재시작합니다.
- **Settings** — Python 버전, Secrets, 공개 범위 등을 나중에 바꿀 수 있습니다.
- **잠자기** — 무료 배포는 한동안 접속이 없으면 앱이 잠듭니다. 접속하면 'Yes, get this app back up!' 버튼이 나오고, 누르면 1분 안에 다시 켜집니다.

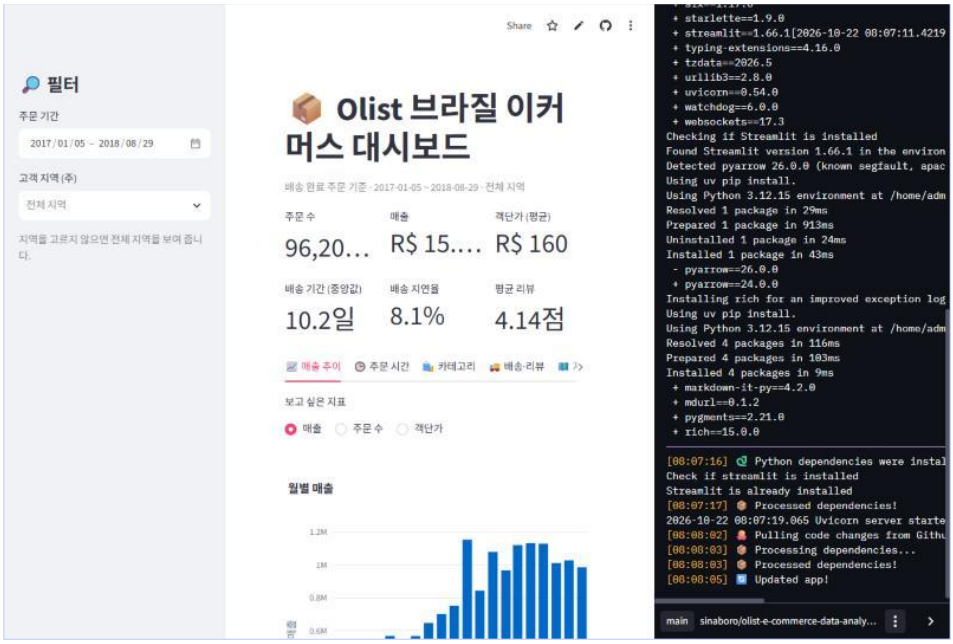


그림 8-12 Manage app 의 로그 — push 하자 'Pulling code changes from Github … Updated app!' 으로 자동 갱신

8.8 README 에 주소 적기

배포 주소를 README.md 의 '배포 주소' 줄에 적고 commit · push 하세요. 저장소를 방문한 사람이 바로 서비스로 들어갈 수 있습니다.

8.9 고치고 다시 배포하기 — push 만 하면 끝

배포된 앱은 GitHub 의 main 브랜치를 지켜보고 있다가 **새 commit 이 push 되면 자동으로 다시 배포**합니다(그림 8-12). 별도로 Deploy 를 다시 누를 필요가 없습니다.

[터미널]

```
git add app.py
git commit -m "대시보드 제목 수정"
git push
```

핵심

배포 = 저장소 · 브랜치(main) · 실행 파일(app.py) · 주소 · Python 버전 5가지 설정. 이후 수정은 commit + push 만 하면 자동 재배포.

■ 확인 문제 (정답은 부록 A)

1. 배포 설정의 Main file path 에 넣는 값과 그 이유는?
2. 배포 중 오류가 났을 때 가장 먼저 확인할 곳은?
3. app.py 를 고친 뒤 배포된 사이트에 반영하려면?

CHAPTER 9

문제 해결

git · GitHub · Streamlit Cloud 에서 자주 만나는 오류와 해결법

학습 목표

- 오류 메시지에서 원인을 찾아 스스로 해결한다.
- 배포 전 체크리스트로 오류를 예방한다.

증상 (메시지)	원인	해결
배포 로그: ModuleNotFoundError: No module named 'plotly'	requirements.txt 에 패키지가 빠짐	requirements.txt 에 추가 → commit → push
FileNotFoundError: data/orders.csv.gz	data 폴더를 push 안 함 / 실행 위치 기준 경로	git add data 후 push / Path(__file__).parent 사용
git push: rejected ... (fetch first)	GitHub 에 내 컴퓨터에 없는 commit 이 있음 (README 를 켜고 만듦 등)	git pull --rebase origin main 후 git push
error: remote origin already exists	remote 를 두 번 추가	git remote set-url origin 새주소
fatal: not a git repository	저장소 폴더 밖에서 git 명령 실행	cd olist_dashboard 로 이동
src refspec main does not match any	commit 을 한 번도 안 하고 push	git add . → git commit 후 다시 push
Authentication failed / 403	로그인 정보가 다르거나 만료	Windows 자격 증명 관리자에서 github.com 항목 삭제 후 다시 push → 로그인
exceeds GitHub's file size limit of 100 MB	큰 파일을 commit	데이터를 줄이거나 압축, .gitignore 에 추가 (이미 commit 했다면 git rm --cached 파일)
Main file path 에 'This file does not exist'	파일 이름 · 위치가 다름	저장소에서 app.py 위치 확인 (하위 폴더면 폴더/app.py)
배포 화면이 계속 'Your app is in the oven'	패키지 설치 중이거나 실패	Manage app 로그 확인 → 오류 메시지대로 수정
App URL 이 'not available'	이미 다른 사람이 쓰는 주소	다른 이름으로 (예: 끝에 내 이름 · 숫자 추가)
'This app has gone to sleep'	오랫동안 접속이 없어 잠듦 (오류 아님)	버튼을 눌러 깨우기

배포 전 체크리스트

- ☐ 로컬에서 streamlit run app.py 가 오류 없이 열린다 (또는 03 노트북의 AppTest 가 '오류: 없음 ✓')
- ☐ requirements.txt 에 app.py 가 import 하는 패키지가 모두 있다 (streamlit · pandas · plotly)
- ☐ 데이터 경로가 Path(__file__).parent 기준이고, data 폴더가 git status 에 보인다
- ☐ 100MB 넘는 파일 · 비밀 정보(secrets.toml) · 가상환경 폴더가 올라가지 않는다
- ☐ 대시보드 하단과 README 에 데이터 출처 · 라이선스가 있다

- ☐ Streamlit Cloud 의 Python 버전을 내 컴퓨터와 맞췄다

■ 확인 문제 (정답은 부록 A)

1. git push 가 rejected (fetch first) 로 거절될 때의 해결 명령은?

2. 배포 로그에 ModuleNotFoundError 가 보이면 어떤 파일을 고쳐야 하나요?

CHAPTER 10

내 도메인 지식으로 확장하기

이 프로젝트를 템플릿 삼아 다른 분야 데이터로 나만의 분석 서비스 만들기

학습 목표

- 이커머스 분석의 구조를 다른 도메인에 옮겨 적용하는 방법을 안다.
- 내 도메인의 질문 · 데이터 · KPI 를 정의한다.
- app.py 를 내 데이터에 맞게 바꾸는 순서를 익히고 미니 프로젝트를 기획한다.

10.1 분석 서비스는 7단계를 반복한다

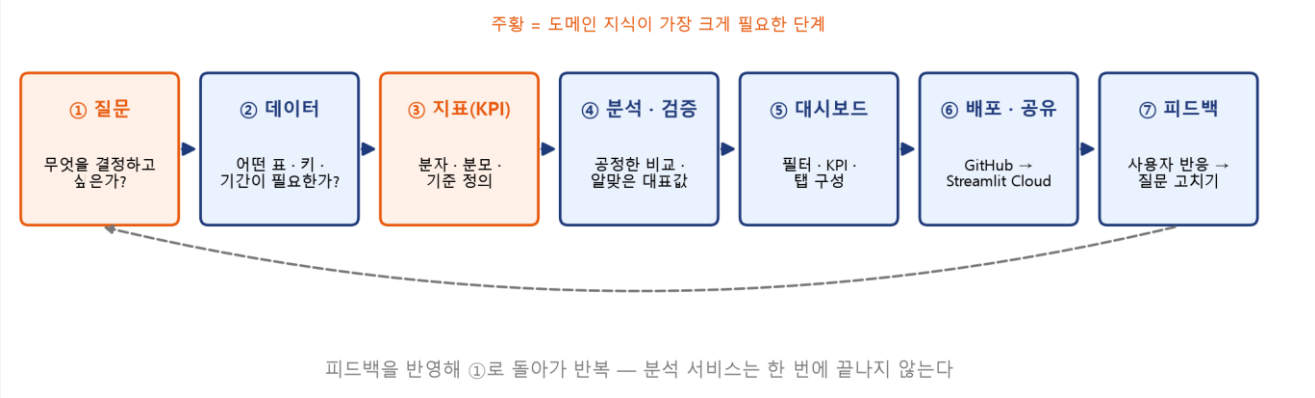


그림 10-1 도메인 분석 서비스의 반복 사이클 — 도메인 지식은 ① 질문과 ③ 지표에서 가장 크게 쓰인다

데이터 분석 기술(pandas · Streamlit · git)은 어느 분야든 같습니다. 분야마다 달라지는 것은 **'무엇을 물어야 하는가(질문)'** 와 **'무엇으로 잘했다고 판단하는가(지표)'** 입니다. 바로 여기에 여러분의 **도메인 지식**이 들어갑니다. 현장 경험이 있는 사람만 '이 숫자가 왜 이상한지', '어떤 비교가 공정한지' 알 수 있습니다.

10.2 이커머스 개념을 일반화하기

이 프로젝트의 테이블과 지표를 한 단계 추상화하면 거의 모든 분야에 그대로 쓸 수 있습니다.

Olist (이커머스)	일반화한 개념	다른 분야에서는
주문 (orders)	거래 · 사건(이벤트) — 언제 · 누가 · 무엇을	수강 신청, 진료 접수, 생산 작업 지시, 민원 접수, 대출 신청
고객 (customer_unique_id)	행동의 주체	학생, 환자, 설비, 민원인, 대출자
상품 · 카테고리	행동의 대상과 분류	강좌 · 과목, 진료과, 제품 라인, 민원 유형, 대출 상품
결제 금액	가치 · 양	수강료, 진료비, 생산량, 처리 비용, 대출액
배송 기간 · 지연	처리 시간(리드타임) · 약속 준수	과제 제출 지연, 대기 시간, 생산 리드타임, 민원 처리 기한 초과

Olist (이커머스)	일반화한 개념	다른 분야에서는
리뷰 점수	결과 · 만족도	강의 평가, 환자 만족도, 불량률, 민원 만족도, 연체율
재구매율	재방문 · 유지(retention)	재수강률, 재진율, 설비 재고장률, 반복 민원 비율, 재대출률

> **핵심**
>
> **'배송 지연 → 리뷰 하락' 은 어느 분야든 '처리 지연 → 만족도 · 결과 하락' 질문으로 바꿔 물을 수 있다.**

10.3 도메인별 확장 예시

도메인	핵심 질문	필요한 데이터	대시보드 KPI · 탭 아이디어
교육 (학원 · 학교)	어떤 학생이 중도 탈락하나? 과제 지연이 성적에 영향을 주나?	수강 기록, 출결, 과제 제출 시각, 성적, 강의 평가	재수강률, 출석률, 과제 지연율, 강좌별 만족도 · 요일별 출석 히트맵
의료 (병원 · 약국)	대기 시간이 길면 만족도가 떨어지나? 어느 진료과가 붐비나?	접수 · 진료 시각, 진료과, 재진 여부, 만족도 설문 (반드시 비식별)	평균 · 중앙 대기 시간, 재진율, 시간대 × 요일 혼잡도
제조 (공장)	어떤 설비 · 교대조에서 불량이 많은가?	작업 지시, 설비 ID, 교대조, 생산량, 불량 수, 정지 시간	불량률, 설비 가동률, 리드타임, 설비별 파레토
물류 · 유통	어느 지역 · 시간대 배송이 늦나?	출고 · 도착 시각, 지역, 차량, 약속 시각	정시 도착률, 지역별 리드타임, 요일 × 시간 물량
금융	어떤 고객군이 연체하나?	대출 신청 · 상환 기록, 고객 속성, 상품	연체율, 상품별 잔액, 코호트별 상환율
공공 · 행정	어떤 민원이 처리 기한을 넘기나?	민원 접수 · 처리 일시, 유형, 담당 부서, 만족도	기한 내 처리율, 유형별 처리 시간, 반복 민원 비율
외식 · 소매 매장	어떤 메뉴가 매출 · 이익을 끌어올리나?	POS 판매 기록, 메뉴, 시간, 할인, 원가	객단가, 시간대 매출, 메뉴 파레토, 재방문율
인사 (HR)	어떤 조건에서 퇴사가 많은가?	입사 · 퇴사일, 부서, 근속, 교육 이수, 만족도 조사	퇴사율, 부서별 근속 중앙값, 교육 이수율

10.4 KPI 정의서 쓰기

지표는 이름만 정하면 사람마다 다르게 계산합니다. 이 프로젝트에서 '주문 수'가 결제 행 수로 잘못 세어질 뻔한 것처럼요. 대시보드를 만들기 전에 아래 표처럼 **분자 · 분모 · 기준**을 글로 확정하세요.

항목	예: 배송 지연율 (이 프로젝트)	내 도메인 지표 (직접 채우기)
지표 이름	배송 지연율	
질문 (왜 보나?)	고객과의 약속을 얼마나 지키나?	

항목	예: 배송 지연율 (이 프로젝트)	내 도메인 지표 (직접 채우기)
분자	실제 배송일 > 예정일 인 주문 수	
분모	배송 완료 주문 수	
대상 · 기간	order_status = delivered, 2017.01~2018.08	
좋은 방향 · 목표	낮을수록 좋음, 목표 5% 이하	
데이터 출처 · 갱신 주기	orders 표, 일 1회	
주의점	성수기 · 지역에 따라 크게 다름 → 월별 · 주별로 함께 볼 것	

10.5 데이터 구하기

출처	주소	특징
공공데이터포털	data.go.kr	정부 · 공공기관 데이터 (교통, 의료, 상권, 민원 등). 파일 다운로드 · API
서울 열린데이터광장	data.seoul.go.kr	서울시 생활 · 교통 · 상권 데이터
KOSIS 국가통계포털	kosis.kr	인구 · 경제 · 사회 통계 (침투율 계산의 분모로 유용)
AI Hub	aihub.or.kr	AI 학습용 대규모 데이터 (텍스트 · 이미지 · 음성)
Kaggle	kaggle.com/datasets	전 세계 공개 데이터 (이커머스, 의료, 금융 등) — 라이선스 확인 필수
회사 · 기관 내부 데이터	-	가장 가치 있지만 반드시 허가 · 개인정보 비식별 처리 후 사용, 공개 저장소 금지

개인정보와 라이선스 — 반드시 지킬 것

- 이름 · 전화번호 · 주민번호 · 상세 주소 등 개인을 알아볼 수 있는 정보는 **삭제하거나 비식별 처리**(번호로 바꾸기, 구간으로 묶기)한 뒤 사용합니다.
- 회사 · 기관 데이터는 **공개(Public) 저장소에 올리지 않습니다.** Streamlit Cloud 의 비공개 앱이나 사내 서버를 쓰고, 담당자의 허가를 받으세요.
- 공개 데이터도 **라이선스**(상업적 이용 가능 여부, 출처 표시 의무)를 확인하고 대시보드에 출처를 적습니다.

10.6 app.py 를 내 데이터로 바꾸는 순서

1. **배포용 데이터 만들기** — 03 노트북 2절처럼 분석 대상을 정하고, 대시보드에 필요한 열만 남겨 data/내데이터.csv.gz 로 저장합니다(긴 ID 는 짧은 번호로).
2. **load_data()** — 파일 이름과 parse_dates(날짜 열 이름)를 바꿉니다.

3. **사이드바 필터** — 기간 필터의 날짜 열, 지역 필터 대신 내 도메인의 범주 열(예: 진료과 · 강좌 · 설비)로 바꿉니다.
4. **KPI 6개** — 10.4 절에서 정의한 지표 계산식으로 바꿉니다(분자 · 분모 그대로 코드로 옮기기).
5. **탭** — 매출 추이 → 내 핵심 지표의 월별 추이, 주문 시간 → 요일 × 시간 패턴, 카테고리 → 대상별 순위, 배송 · 리뷰 → 처리 시간 vs 결과, 지역 → 내 범주 비교.
6. **제목 · 출처 · README** — 페이지 제목, 하단 출처, README 의 설명 · 인사이트를 내 내용으로.
7. **테스트 → push → 배포** — 6장 · 7장 · 8장과 같은 순서로 새 저장소에 올리고 배포합니다.

[코드] KPI 를 내 도메인 지표로 바꾼 예

```
# 예: 병원 대기 시간 대시보드로 바꿀 때의 KPI 부분
k1.metric("접수 건수", f"{len(f_df):,}건")
k2.metric("대기 시간 (중앙값)", f"{f_df['wait_min'].median():.0f}분")
k3.metric("30분 초과 비율", f"{(f_df['wait_min'] > 30).mean() * 100:.1f}%")
k4.metric("재진율", f"{f_df['revisit'].mean() * 100:.1f}%")
k5.metric("평균 만족도", f"{f_df['satisfaction'].mean():.2f}점")
```

10.7 미니 프로젝트 — 나만의 분석 서비스

이 교안의 마지막 과제입니다. 자신이 일했거나 관심 있는 분야의 데이터로 **질문 → 분석 → 대시보드 → 배포**까지 완성하고, 서비스 주소를 제출합니다.

단계	제출물	체크 포인트
1. 기획	기획서 1장: 질문 3개, 데이터 출처, KPI 정의서(10.4) 2개 이상	질문이 '결정'과 연결되는가?
2. 분석	분석 노트북 (실행 결과 포함)	분석 대상 정의, 공정한 비교, 알맞은 대표값
3. 인사이트	인사이트 문장 3개 이상 (사실 · 근거 · 의미 · 제안)	각 인사이트의 주의점(한계) 포함
4. 대시보드	app.py, requirements.txt, 가공 데이터	필터 1개 이상, KPI 3개 이상, 탭 2개 이상
5. 배포	GitHub 저장소 주소 + streamlit.app 주소, README	README 에 서비스 주소 · 실행 방법 · 데이터 출처

평가 항목	배점	기준
질문과 KPI 의 명확성	20	도메인 맥락이 분명하고 분자 · 분모가 정의됨
분석의 정확성	25	merge · 집계 · 비교 기간이 올바르고 근거가 재현 가능
인사이트의 질	25	숫자 → 의미 → 실행 제안까지 연결, 한계 언급
대시보드 완성도	20	필터 · KPI · 탭이 동작하고 읽기 쉬움, 출처 표시
배포와 문서화	10	주소로 접속 가능, README 충실, 비밀 정보 · 개인정보 없음

10.8 더 나아가기

- **예측 탭 추가** — 05 로지스틱 회귀 · 06 KNN 에서 배운 모델로 '지연될 주문'이나 '이탈할 고객'을 예측하는 탭을 만들 수 있습니다(모델은 미리 학습해 파일로 저장해 두고 app.py 에서 불러오기).
- **지도 시각화** — 지역 데이터에 위도 · 경도가 있다면 st.map 이나 plotly 의 지도 그래프로 지역 격차를 한눈에 보여 줍니다.
- **데이터 자동 갱신** — 데이터 파일만 새로 만들어 push 하면 대시보드가 자동으로 최신화됩니다. 정기적으로 갱신하는 스크립트를 만들어 보세요.
- **비밀 정보가 필요한 앱** — API 키(예: ChatGPT)를 쓰는 기능은 코드에 키를 적지 말고 Streamlit Cloud 의 **Secrets** 에 넣고 st.secrets["키이름"] 으로 읽습니다.

■ 확인 문제 (정답은 부록 A)

1. '배송 지연 → 리뷰 하락' 질문을 병원 도메인으로 바꾸면?
2. KPI 정의서에서 반드시 적어야 하는 세 가지는?
3. 회사 데이터로 대시보드를 만들 때 주의할 점 두 가지는?

APPENDIX A

부록 A. 확인 문제 정답

각 장 끝 확인 문제의 정답과 짧은 풀이

1장 프로젝트 한눈에 보기

Q 대시보드가 원본 데이터(압축 44MB) 대신 2.4MB 짜리 가공 데이터를 읽는 이유 두 가지는?

A ① 무료 배포 서버에서 큰 파일을 접속할 때마다 읽으면 느리고 메모리도 많이 쓰기 때문 ② 대시보드에는 분석이 끝난 몇 개의 열만 있으면 되기 때문. (GitHub 는 파일 하나가 100MB 를 넘으면 올릴 수 없고 50MB 부터 경고하는 제한도 있다.)

Q Streamlit Cloud 는 어디에 있는 코드를 실행하나요?

A GitHub 저장소에 올라간 코드(app.py)를 가져와 실행한다. 그래서 GitHub 에 push 해야 배포에 반영된다.

2장 실습 환경과 데이터 준비

Q 노트북 코드의 PATH = "../data/" 에서 .. 은 무엇을 뜻하나요?

A 현재 폴더의 '한 단계 위(부모) 폴더'. notebooks 폴더의 상위(저장소 최상위)에 있는 data 폴더를 가리킨다.

Q 원본 데이터를 .csv 가 아니라 .csv.gz 로 저장소에 두는 이유는?

A 용량을 126MB → 44MB 로 줄여 GitHub 의 50MB 경고를 피하고 clone · 배포를 빠르게 하기 위해. pd.read_csv 는 .csv.gz 를 그대로 읽는다.

Q Olist 데이터를 사용할 때 지켜야 할 라이선스 조건 세 가지는?

A 비영리(NC), 출처 표시(BY), 같은 조건으로 공유(SA).

3장 데이터 구조 이해하기

Q 고객 수를 셀 때 customer_id 가 아니라 customer_unique_id 를 쓰는 이유는?

A customer_id 는 주문할 때마다 새로 생겨 주문 수와 같아지기 때문. 실제 사람은 customer_unique_id 로 구분한다.

Q orders 와 payments 를 그대로 merge 한 뒤 행을 세면 무엇이 세어지나요?

A 결제 행 수. 결제를 여러 번 한 주문이 여러 행으로 복제되기 때문이다.

Q 이 프로젝트에서 '매출'로 보는 주문의 조건은?

A order_status 가 delivered(배송 완료)인 주문, 금액은 그 주문의 payment_value 합계.

4장 분석 핵심 기법과 흔한 실수

Q 월별 매출 그래프에서 막대가 한 달씩 밀려 보인다면 무엇을 확인해야 하나요?

A 월 묶기 freq 가 ME(월말)인지 확인하고 MS(월초)로 바꾼다.

Q 주문 금액의 대표값으로 평균보다 중앙값이 나은 이유는?

A 주문 금액은 오른쪽 꼬리가 긴 분포라 큰 주문 몇 건이 평균을 끌어올리기 때문(평균 160 > 중앙값 105).

5장 주요 인사이트 10가지

Q 인사이트 6에서 분기 합계를 모든 연도로 섞어 계산하면 4분기가 낮게 나오는 이유는?

A 2018년 4분기 데이터가 없고 2016년 4분기는 거의 0이라, 4분기에 들어간 기간이 다른 분기보다 적기 때문.

Q 인사이트 9(배송 지연과 리뷰)의 주의점은?

A 강한 관계(상관)이지만 인과라고 단정하려면 상품 · 판매자 등 다른 요인을 통제한 추가 분석이 필요하다.

Q 인사이트를 문장으로 쓸 때의 4단계 순서는?

A 사실 → 근거 → 의미 → 제안

6장 Streamlit 대시보드 이해하기

Q @st.cache_data 를 쓰지 않으면 어떤 문제가 생기나요?

A 필터를 바꿀 때마다 app.py 전체가 다시 실행되면서 CSV 를 매번 다시 읽어 화면이 느려진다.

Q 데이터 경로에 Path(__file__).parent 를 쓰는 이유는?

A app.py 가 있는 폴더를 기준으로 파일을 찾아, 어느 위치에서 실행하든(배포 서버 포함) 경로가 맞게 하기 위해.

7장 GitHub 에 올리기

Q 고칠 때마다 반복하는 git 명령 3단계는?

A git add → git commit -m "메시지" → git push

Q 새 저장소를 만들 때 Add README 를 켜면 어떤 문제가 생길 수 있나요?

A GitHub 저장소에 내 컴퓨터에 없는 commit(README)이 생겨 첫 push 가 rejected(fetch first) 로 거절될 수 있다.

Q git init 을 olist_dashboard 폴더 안에서 해야 하는 이유는?

A 상위 폴더에서 하면 원본 데이터와 노트북 등 올리지 않을 파일까지 저장소에 포함되기 때문.

8장 Streamlit Cloud 로 배포하기

Q 배포 설정의 Main file path 에 넣는 값과 그 이유는?

A app.py — 저장소 최상위에 있는 대시보드 실행 파일이 app.py 이기 때문(기본값 streamlit_app.py 가 아님).

Q 배포 중 오류가 났을 때 가장 먼저 확인할 곳은?

A 대시보드 오른쪽 아래 Manage app 의 로그(설치 과정과 오류 메시지).

Q app.py 를 고친 뒤 배포된 사이트에 반영하려면?

A git add → git commit → git push. Streamlit Cloud 가 새 commit 을 감지해 자동으로 다시 배포한다.

9장 문제 해결

Q git push 가 rejected (fetch first) 로 거절될 때의 해결 명령은?

A git pull --rebase origin main 을 실행한 뒤 다시 git push

Q 배포 로그에 ModuleNotFoundError 가 보이면 어떤 파일을 고쳐야 하나요?

A requirements.txt — 빠진 패키지 이름을 추가하고 commit · push

10장 내 도메인 지식으로 확장하기

Q '배송 지연 → 리뷰 하락' 질문을 병원 도메인으로 바꾸면?

A '진료 대기 시간이 길수록 환자 만족도가 떨어지는가?' (처리 지연 → 만족도 하락)

Q KPI 정의서에서 반드시 적어야 하는 세 가지는?

A 분자, 분모, 대상 · 기간(기준). 그 밖에 좋은 방향 · 목표와 주의점.

Q 회사 데이터로 대시보드를 만들 때 주의할 점 두 가지는?

A 개인정보를 삭제 · 비식별 처리하고, 공개 저장소 · 공개 앱에 올리지 않는다(허가 · 비공개 배포).

APPENDIX B

부록 B. 명령어 · 코드 한 장 요약

책상 옆에 두고 보세요

분류	명령 · 코드	하는 일
설치	pip install pandas plotly streamlit jupyter	실습 패키지 설치
실행	streamlit run app.py (끝내기: Ctrl + C)	대시보드를 내 컴퓨터에서 실행
git 처음	git config --global user.name / user.email	commit 작성자 등록 (한 번)
git 처음	git init → git branch -M main → git remote add origin 주소	저장소 만들고 GitHub 연결 (한 번)
git 반복	git add . → git commit -m "메시지" → git push	변경 기록하고 올리기 (자동 재배포)
git 확인	git status · git log --oneline · git remote -v	상태 · 기록 · 연결 주소 확인
git 해결	git pull --rebase origin main	push 가 rejected 될 때
pandas	df["키"].is_unique	merge 전 키 중복 확인
pandas	groupby(...).agg(이름=("열", "계산"))	여러 집계를 한 번에
pandas	pd.Grouper(key="날짜", freq="MS")	월 단위 묶기 (그래프용 월초)
Streamlit	@st.cache_data	데이터 읽기 결과 캐시
Streamlit	st.sidebar.date_input / multiselect / radio / slider	필터 · 선택 위젯
Streamlit	st.columns · st.metric · st.tabs · st.plotly_chart	배치 · KPI · 탭 · 그래프
Streamlit	st.download_button · st.secrets["키"]	파일 내려받기 · 비밀 정보 읽기

APPENDIX C

부록 C. 참고 링크

이 교안에서 사용한 주소 모음

무엇	주소
예시 GitHub 저장소	https://github.com/sinaboro/Olist-E-Commerce-Data-Analysis
예시 대시보드 (서비스)	https://olist-ecommerce-sinaboro.streamlit.app
Olist 데이터 (Kaggle)	https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce
Streamlit Community Cloud	https://share.streamlit.io
Streamlit 공식 문서	https://docs.streamlit.io
plotly 그래프 예제	https://plotly.com/python
git 설치	https://git-scm.com
GitHub	https://github.com